USER / DEVELOPER GUIDE

# Integrating 3rd party websites with ADvance

Version 1.2.5

09/10/2023

abacus

# Table of Contents


1 Introduction ..... 6

2 Linking from your website to AD pages ..... 6

2.1 List of AD frontend pages ..... 6

2.2 Branding and messaging on AD pages ..... 7

2.3 Shared root domain ..... 7

2.4 SSL secure certificates ..... 8

3 Interacting with AD APIs ..... 8

3.1 How to connect to the AD API ..... 8

3.1.1 API HOST and API Token ..... 8

3.1.2 API Hash key ..... 8

3.1.3 Testing connection to AD API ..... 8

3.2 URL encoding ..... 9

3.3 Sample code to call an AD API (AD Helper method) ..... 9

3.4 List of AD APIs available ..... 11

4 Data stored in AD versus data stored locally ..... 11

5 Registration ..... 11

5.1 What you need to do? ..... 11

5.2 Registration workflow ..... 13

5.3 Post-registration tasks ..... 14

5.3.1 Sign-in following registration & obtaining 'UserToken' ..... 14

5.3.2 Creating a stub record ..... 14

6 Sign-in ..... 15

6.1 Introduction ..... 15

6.2 Login methods ..... 15
6.2.1 Redirect based Login process ..... 15
6.2.1.1 Introduction ..... 15
6.2.1.2 Workflow diagram ..... 16
6.2.2 API based Login process ..... 16
6.2.2.1 Introduction ..... 16
6.2.2.2 Workflow diagram ..... 17
6.2.2.3 Code sample to perform standard Login ..... 18
6.3 AutoLogin process ..... 18
6.3.1 Introduction ..... 18
6.3.2 Workflow diagram ..... 19
7 Email activation (optional) ..... 19
8 Enabling single-sign-on with AD pages after a successful website login ..... 20
8.1 SSO cookie set via a redirect to the AD cookie creation page ..... 21
8.2 SSO cookie set via JavaScript initiated redirect to the AD cookie page ..... 21
8.3 SSO cookie set on first access to an AD page ..... 21
9 Requesting user details ..... 21
9.1 Introduction ..... 21
9.2 Workflow diagram ..... 22
9.3 SimpleUserDetails API ..... 22
9.4 Code sample to request user details ..... 22
10 Edit user details ..... 23
10.1 Introduction ..... 23
10.2 Seamless access to AD 'Edit user details' page ..... 23

10.3 Updating local website database ..... 23

11 Password recovery ..... 24

11.1 Introduction ..... 24

11.1.1 Testing password recovery API integration ..... 24

11.2 ForgottenPassword API ..... 25

11.2.1 Calling the API ..... 25

11.2.2 Code sample to invoke the ForgottenPassword API ..... 25

11.2.3 Frontend user experience ..... 25

11.3 PasswordReset API ..... 27

11.3.1 Frontend user experience ..... 27

12 Implementing gatekeeping on premium online content ..... 29

12.1 Introduction ..... 29

12.2 Requesting access for individual content items based on Metadata/category rules ..... 29

12.2.1 Introduction ..... 29

12.2.2 Workflow diagram ..... 29

12.2.3 Calling the API ..... 30

12.2.4 Supporting endpoints required to be provided by the non-Abacus content management platform ..... 30

12.2.4.1 Category/taxonomy schema endpoint ..... 30

12.2.4.2 Article/story category associations ..... 31

12.3 Requesting access based on URL of the item to be gated ..... 32

12.3.1 Introduction ..... 32

12.3.2 Workflow diagram ..... 32

12.3.3 Calling the API ..... 33

12.4 Code sample for making a gatekeeping call ..... 33

12.5 Retrieving details about a user's subscription products ..... 33
12.5.1 Introduction ..... 33
12.5.2 Workflow diagram ..... 34
12.5.3 Business rules ..... 34
12.5.4 Calling the API ..... 34
13 Access denied messaging ..... 34
13.1 Introduction ..... 34
13.2 Gatekeeping levels ..... 34
13.3 Frontend user experience on your website ..... 35
13.3.1 Example 1 ..... 35
13.3.2 Example 2 ..... 36
14 Selling subscriptions on your website using AD ..... 36
14.1 Introduction ..... 36
14.2 Linking out to AD subscription pages ..... 36
14.3 Subscription / checkout steps ..... 37
14.4 Website integration ..... 38
15 Managing newsletter sign-ups ..... 39
16 Other user messaging options available to you ..... 40
16.1 Frontend online renewal messaging ..... 40
16.2 Other ad-hoc messaging ..... 41
17 Appendix: AD call method helper classes ..... 41
17.1 ADResult ..... 41
17.2 ADResponseFormat ..... 46
17.3 ADMethod ..... 47

# 1 Introduction

The purpose of this document is to issue guidelines to 3rd party developers who want to integrate their websites with Abacus's Advance (AD) platform.

# 2 Linking from your website to AD pages

For all user account-related tasks like registering, updating user details or purchasing a subscription, please add a hyperlink from your website to the corresponding AD page. A list of available AD pages has been provided below for your reference.

The core operation like registration or purchase will be handled on AD pages and the user will be passed back to your website (depending on the return URL set). You will need to make use of a common cookie and make the necessary API calls to AD to read information about the user for use on your website.

## 2.1 List of AD frontend pages

The following AD frontend pages are available. To access them you need to add the given suffix to the AD Frontend URL. For example, if you want to access the default registration form you need to append the AD frontend URL, i.e. https://ad.thelawyer.com with the relevant suffix listed in the table below **/register** to get https://ad.thelawyer.com/register

| Suffix | Purpose / examples |
|---|---|
| **/subscribe** | Product Offers page listing all the different subscriptions options available to a user https://ad.thelawyer.com/subscribe<br><br>https://ad.thelawyer.com/subscribe**?sourcecode=TLUK20014** (to invoke a specific Product Offers page identified by a source code) |
| **/login** | Standalone sign-in page, i.e. http://account.ipe.com/login |
| **/signin** | Checkout sign in page |
| **/eudsignin** | Sign in page automatically shown when a logged out user is trying to access the Edit user details form |
| **/mssignIn** | Sign in page automatically shown when a logged out user is trying to access the My Subscriptions page |
| **/forgottenpassword** | Forgotten password page for password recovery https://ad.thelawyer.com/ForgottenPassword |
| **/SubmitResetPasswordRequest** | Page where the end user can generate a reset password request, i.e. https://account.ipe.com/SubmitResetPasswordRequest |
| **/ResetPassword** | This is the page shown when user clicks on the reset password link in the password reset email |

| | |
|---|---|
| **/register** | Default registration form https://ad.thelawyer.com/register<br><br>https://account.ipe.com/register**?surveycode=11** (to invoke a registration form different to the one marked as default)<br><br>Example with a return URL:<br>https://account.ipe.com/register**?returl=http%3a%2f%2fhub.ipe.com** (user will be directed to the return URL specified after completing the form) |
| **/thankyou** | Thank you page shown at the end of a flow if the return URL is unknown. For e.g., after a user has successfully submitted a registration form and no return URL was specified the thank you page is shown (https://account.pharmaceutical-journal.com/thankyou) |
| **/edituserdetails** | Edit user details (also known as Edit my account) form https://account.pharmaceutical-journal.com/EditUserDetails |
| **/mysubscriptions** | My subscriptions page https://account.pharmaceutical-journal.com/mysubscriptions |
| **/gathering** | Incremental data gathering form https://subs.insidehousing.co.uk/gathering?surveycode=17&returl=http://www.insidehousing.co.uk/my-account<br><br>This enables a publisher to collect user details in exchange of granting them access to premium content. Publishers could set up a tiered online gating model, i.e. get 4 clicks free, then register, then 4 more clicks and then fill in a data gathering form to get limited or unlimited access to gated content. |
| **/questionnaire** | This allows user to fill in any form. If user isn't logged in, this flow will ask them to log in prior to filling the form out. |
| **/registrationselection** | Controlled Circulation http://ad.thelawyer.com/registrationselection |
| **/shop** | Only when the e-commerce shop has been configured and set up https://catalogues.christies.com/christies-shop/ |

## 2.2 Branding and messaging on AD pages

AD pages can have the branding (masthead/footer) of your website. Abacus AD services team can help you set it up.

AD pages also have the ability to display custom text so that you can show content relevant to your website/brand. Please talk to the AD services team to know more.

## 2.3 Shared root domain

Please note the AD frontend pages and your website must be on the same domain for all features to work correctly and seamlessly.

| Website URL | AD URL | Allowed |
|---|---|---|
| www.ipe.com | **account**.ipe.com | Yes |
| www.ipe.com | **account**.internationalpublishers.co.uk | No |

| | | |
|---|---|---|
| www.internationalpublishers.co.uk | **account**.ipe.com | No |
| www.internationalpublishers.co.uk | **account**.internationalpublishers.co.uk | Yes |

**IMPORTANT:** Please change your development website URL to match the AD test domain (abasoft.co.uk) using your hosts file, for e.g. websitedev.abasoft.co.uk. Alternatively, you can ask us to change the AD test URL to match your development website domain. This will allow you to run and test the cookie integration code.

### 2.4 SSL secure certificates

We require all AD frontend pages on LIVE to run over secure certificates. Abacus's Sysman team will work with you to procure and install the SSL certificates.

The TEST environment does not need to use secure certificates.

# 3 Interacting with AD APIs

For website features like signing in or requesting forgotten passwords you can continue to use your website pages if you wish to. Some clients prefer to do this so as to keep the user within their website furniture and also because this allows them to exercise greater control over messaging and display, i.e. if you want the username and password input fields to display within a popup, it has to be done via a website page as AD pages don't work well within a pop-up.

Please note that should you wish to use your own pages for sign in or forgotten password, it needs to be programmed to interact with AD by calling the appropriate API.

### 3.1 How to connect to the AD API

#### 3.1.1 API HOST and API Token

This information is common across all brands. You will need this to connect to our APIs.

| Element | TEST URL | LIVE URL |
|---|---|---|
| API HOST | http://adgate.abasoft.co.uk | https://adapi.abacusemedia.com |
| API Token | *Abacus will generate and supply when requested* | *Abacus will generate and supply when requested* |

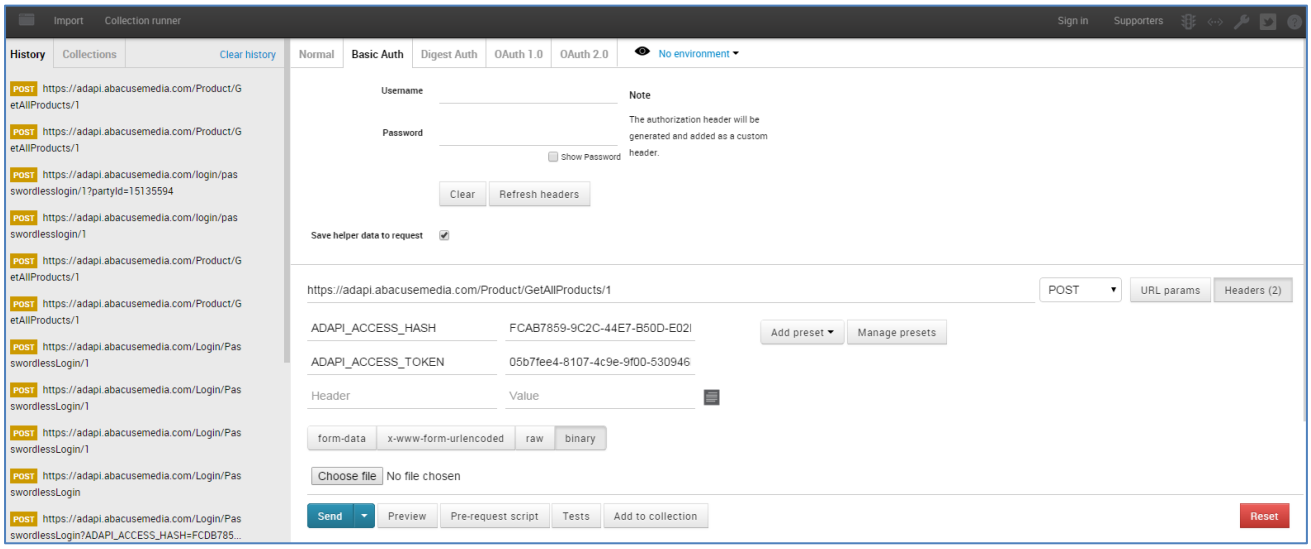
#### 3.1.2 API Hash key

You will need an API Hash Key per brand/website. *Abacus will generate and supply these when requested.* You will need this to connect to our APIs.

#### 3.1.3 Testing connection to AD API

Abacus staff have access to an internal AD API tester tool for the TEST environment. In case you are having issues, please let us know the brand you are trying to connect to and we can check it out our end to check if a connection can be established and that API calls can be successfully made.

Alternatively, you can use tools like the Google Chrome **POSTMAN** to test connection to the AD API. This tool is currently available free of charge.




## 3.2 URL encoding

AD APIs need to be called via HTTP requests. All parameters being passed to an AD API must form a valid URL request and therefore they should be URL-encoded. This is to ensure that any 'invalid' characters do not fail calls to the AD API. Most coding languages have a native URL encoder so it should be fairly simple to add the encoding function.

## 3.3 Sample code to call an AD API (AD Helper method)

Below is the AD Helper method that you may use.

```
  public static ADResult Invoke(string accessHash, string accessToken, ADResponseFormat
responseFormat, string url, ADMethod method, NameValueCollection parameters)
        {
            HttpWebRequest request = null;
            WebResponse response = null;

            try
            {
                StringBuilder body = new StringBuilder();

                bool needsSep = false;

                foreach (string key in parameters.AllKeys)
                {
                    if (needsSep)
                        body.Append('&');
                    else
                        needsSep = true;

                    body.Append(key);
                    body.Append("=");
                    body.Append(HttpUtility.UrlEncode(parameters[key]));
                }
```

```
                if (method == ADMethod.GET)
                    url = url + "?" + body.ToString();

                request = (HttpWebRequest)HttpWebRequest.Create(url);

                // set security context
                request.Headers["ADAPI_ACCESS_HASH"] = accessHash;
                request.Headers["ADAPI_ACCESS_TOKEN"] = accessToken;
                request.Headers["ADAPI_RESPONSE_FORMAT"] = Convert.ToString(responseFormat);

                // method
                request.Method = Convert.ToString(method);

                // if post, build the request body
                if (request.Method.Equals(Convert.ToString(ADMethod.POST)))
                {
                    request.ContentType = "application/x-www-form-urlencoded; charset=UTF-8";

                    byte[] requestBody = Encoding.UTF8.GetBytes(body.ToString());
                    request.ContentLength = requestBody.Length;

                    using (Stream stream = request.GetRequestStream())
                    {
                        stream.Write(requestBody, 0, requestBody.Length);
                        stream.Flush();
                    }
                }

                request.Timeout = GetRequestTimeout();
                response = request.GetResponse();
                ADResult result = null;


                using (Stream stream = response.GetResponseStream())
                {
                    StreamReader reader = new StreamReader(stream, Encoding.UTF8);
                    String responseString = reader.ReadToEnd();
                    LogHelper.Log(string.Format("AD RESPONSE : {0}", responseString), 10);

                    if (responseFormat == ADResponseFormat.XML)
                    {
                        using (XmlTextReader xmlReader = new XmlTextReader(new
System.IO.StringReader(responseString)))
                        {
                            result = new ADResult();
                            result.ReadXml(xmlReader);
                        }
                    }
                    else if (responseFormat == ADResponseFormat.JSON)
                    {
                       //Do nothing.... as JSON calls won't be made from server.
                    }
                    else
                        throw new ArgumentException(string.Format("Invalid AD response type
found: {0}", Convert.ToString(responseFormat)));
                }

                response.Close();
```

```
            return result;
        }
        catch(Exception ex)
        {
            //An error has occurred please log and return empty AD result.
            return new ADResult();
        }
        finally
        {
            if (response != null)
                response.Close();
        }
    }
```

The following classes mentioned in the sample code above will be needed. Please see **Section 17** for more details.

- ADResult
- ADResponseFormat
- ADMethod

### 3.4 List of AD APIs available

Please ask your Account/Project Manager for the latest **AD API document** if you haven't already been given a copy. This contains the full list of the APIs we offer along with their request and response formats.

# 4 Data stored in AD versus data stored locally

Unless specified otherwise, **AD will be the master database**. All user account-related data like names, addresses, username/passwords, demographics etc. must be stored in AD.

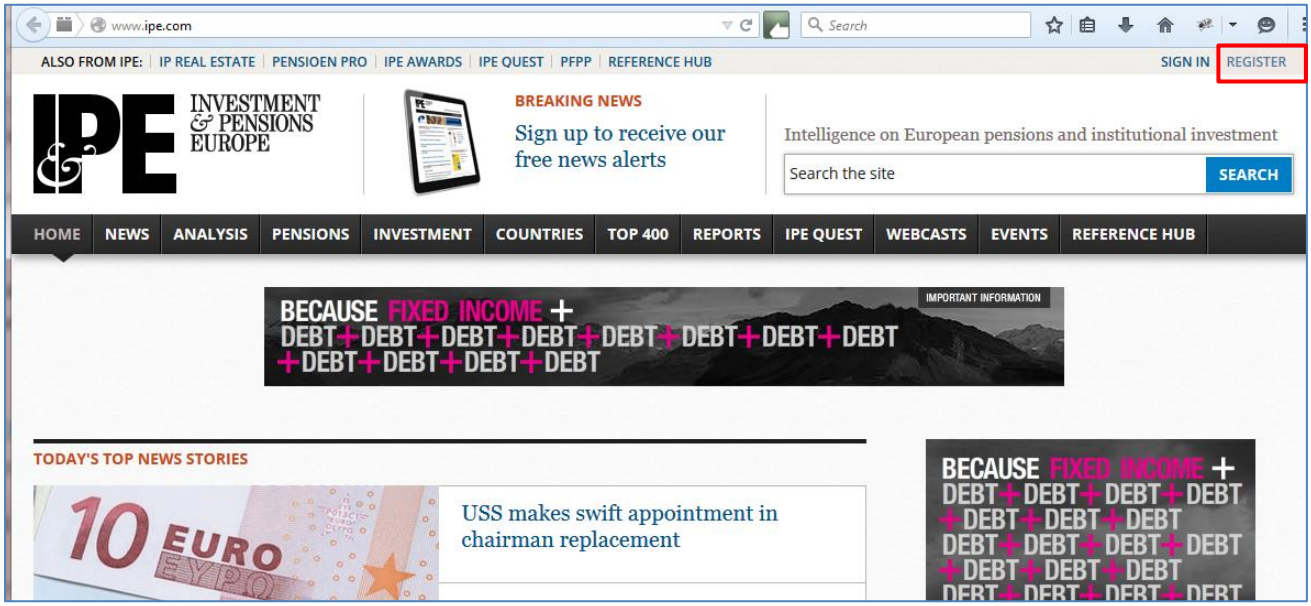
If your website has specific features like allowing users to post comments on articles/forums/blogs, start discussion threads, create a display name, save posts and articles to their reading list, rate articles etc., you should keep all associated data in your website database. AD does not need to store this information.

# 5 Registration

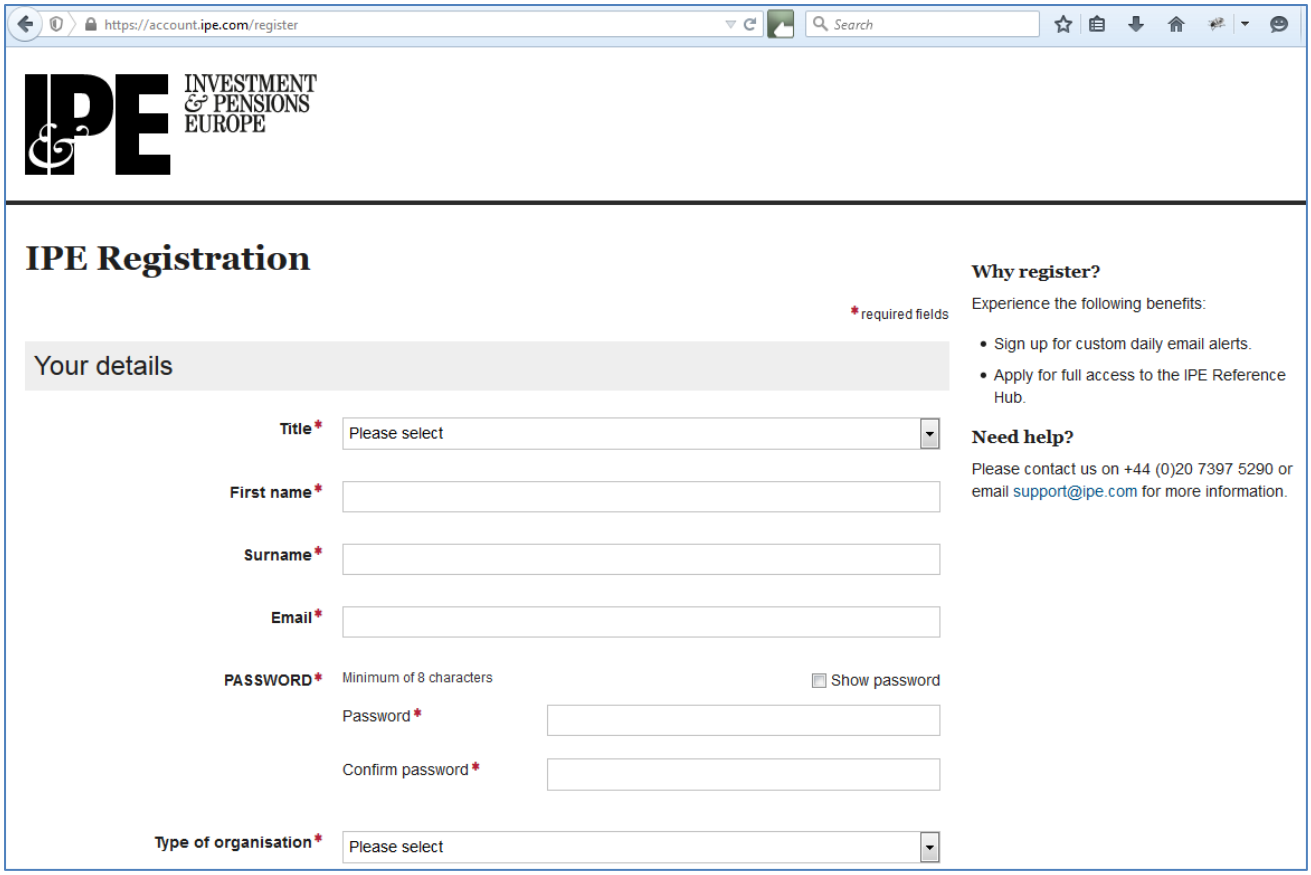
### 5.1 What you need to do?

Registration should always happen using one of the AD capture forms. We strictly recommend against using your own forms. AD has a dynamic form generation engine which is quite powerful yet easy to use.

If your website requires/allows users to register, you need to add a hyperlink from your website to the AD registration page.



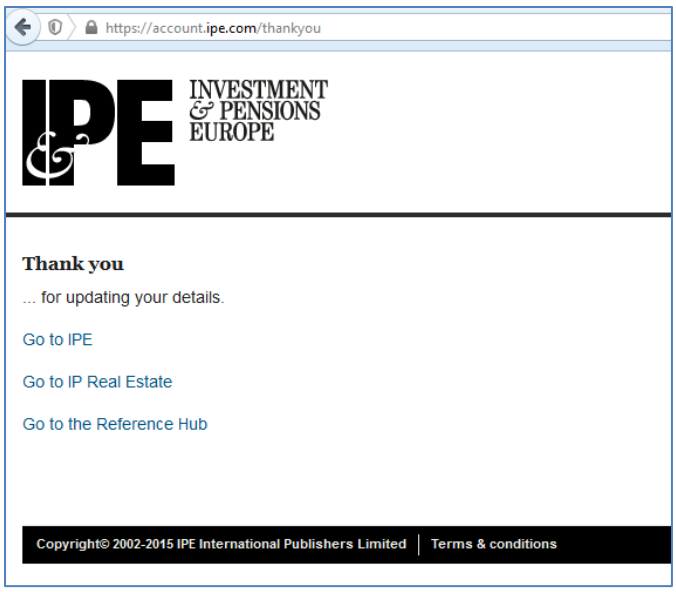

The AD registration form looks like the following.



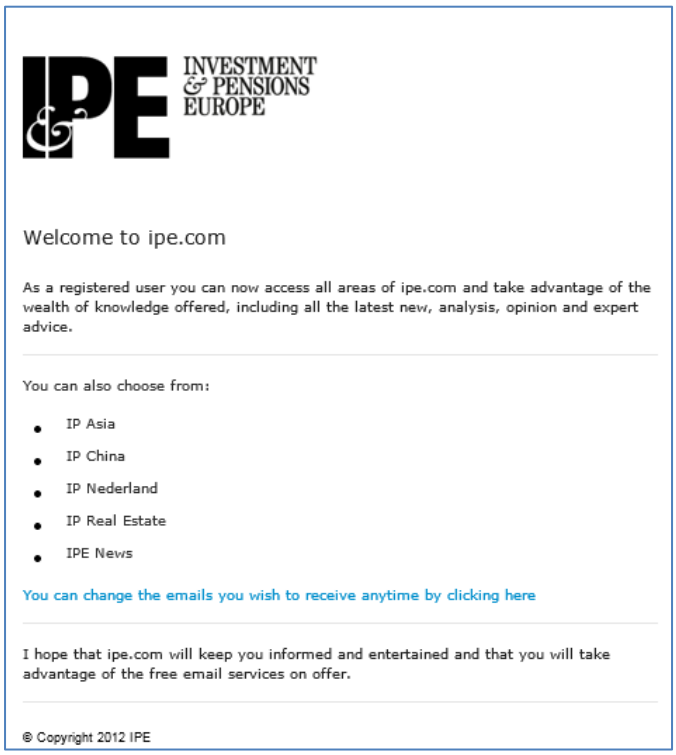

Any existing website registration form that you have needs to be decommissioned once the AD form is up and running.

You may or may not specify a return URL. If you do, the user will be taken to the URL specified after completing the form. If you don't they will see the default AD thank you page.

Contents of the thank you page are driven by an AD test asset. Please ensure this is correctly populated. Abacus AD services team can assist you with this.




After completing the registration form, the user will get a confirmation email. This is also driven by another AD text asset. Please ensure this is correctly populated.

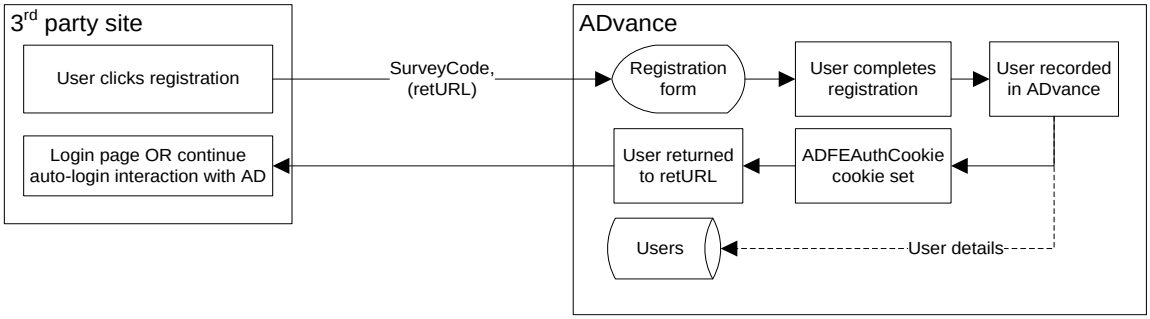



## 5.2 Registration workflow

- User clicks on **Register** link on your website
- User is taken to the AD registration page

# User registers successfully

- AD creates 'ADFEAuthCookie' cookies on the root domain
- User is sent back to your website (where a retURL is supplied, otherwise the AD registration completion page is shown with a link to the originating website)
- Website should detect 'ADFEAuthCookie' cookie[1] and optionally carry out an auto-login action. The AD auto login process is detailed in **Section 6.3**




[1] The cookie can only be used if the AD pages and the website share the same root domain.
The cookie is set with 'httponly' meaning it cannot be read by Javascript methods. This cookie must be read server-side.

## 5.3 Post-registration tasks

### 5.3.1 Sign-in following registration & obtaining 'UserToken'

Following the completion of the AD registration process a user will not be signed in to your website, nor will a 'UserToken' be automatically available. The 'UserToken' is a mandatory parameter that needs to be passed while invoking quite a few of the AD APIs.

To obtain the 'UserToken' and log the user in, one of the following needs to be done.

- The 'AutoLogin' method will need to be called to get the 'UserToken' value
- The user can also be presented with a login page on your website. Once they complete the standard login process via the API, 'UserToken' will be returned.

AD login methods are detailed in **Section 6** of this document.

### 5.3.2 Creating a stub record

For new registrations, you should create a stub record in your local database by fetching user details from AD via an API call. This should serve as a unique link between the AD database and your website database and help manage future updates to that user record.

**It is strongly recommended that AD's unique identifier for the user (i.e. 'Party ID') is stored on your website following the completion of the registration process. This value is returned in the 'AutoLogin' and 'Login' method as the element labelled 'PartyID'.**

# 6 Sign-in

## 6.1 Introduction

AD recommends the use of email address as a party's username and, although it is possible, we do not recommend having a separate non-email username field.

We also offer another sign mode using the party's URN number (Party ID) and surname. If you need this, please speak to us.

Only one mode can be enabled at a time.

We recommend you refresh the local website user record in your database at each login.

We also offer single-sign-on across multiple sites using AD which will allow the user to login to multiple sites using the same set of credentials. Please speak to us and we can discuss this in detail with you.

## 6.2 Login methods

Two login methods are recommended for 3rd party integrations:

- A 'Redirect based Login process' which is the simplest option to implement, but has a requirement that the 3rd party website resides on the same main domain as the ADvance frontend website.
- An 'API based login process' which allows for greater flexibility in completing authentication.

Both methods are detailed within this section.

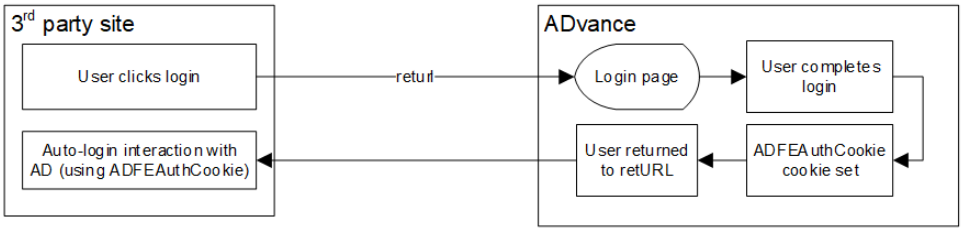
### 6.2.1 Redirect based Login process

#### 6.2.1.1 Introduction

This method involves the redirection of users to the AD frontend to complete their login process, after which they will automatically be returned to a URL specified in the original redirection request.

- Note:
    - This method relies on the consumption of an authentication cookie set within the ADvance frontend domain, and so requires the 3rd party site to also reside on this same domain. If this is not the case, please refer to the alternative API based method detailed in **Section 6.2.2**.
    - The cookie is set with 'httponly' meaning it cannot be read by Javascript methods. **This cookie must be read server-side**.

#### 6.2.1.2 Workflow diagram




This method requires the 3rd party site to direct the user to the AD frontend 'login page' URL, supplying a 'returl' GET parameter that specifies where the user should be returned to following a successful login.

The login page URL would be requested like the following example (with your post login handler included as the 'returl' parameter, with this being URLencoded):

- http://account.yourADfrontendsite.com/login?returl=https%3A%2F%2Fwww.3rdPartySite.com%2F3rdPartyPostLoginHandler%2F

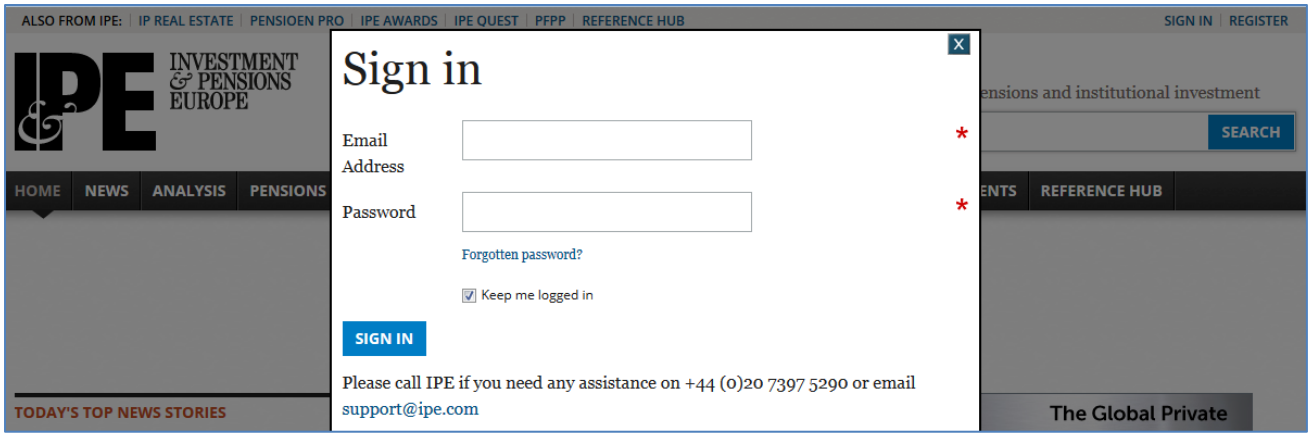
Following a successful sign in via this process, the user will be returned to your site with an 'ADFEAuthCookie' cookie set. This can be used in the 'AutoLogin' process as detailed in **Section 6.3**.

### 6.2.2 API based Login process

#### 6.2.2.1 Introduction

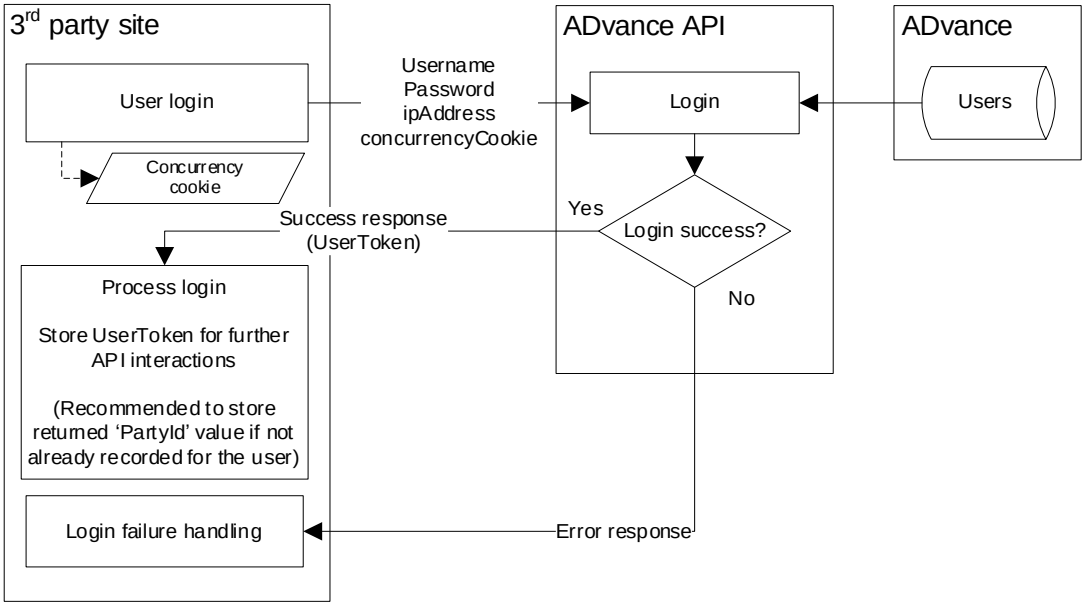
The API based sign in method is a method used for authenticating a user within AD using username and password details. This is to be used where you wish to show a login prompt within your own website that you will then connect to authenticate the user via AD's API methods.

- Note: *An alternative method to achieve a login state immediately following a registration flow completion is detailed in **Section 6.3***.



#### 6.2.2.2 Workflow diagram

The diagram below details the standard login process and the required interactions with the AD API.




| # | 3rd party website action | ADvance API interaction | ADvance action |
|---|---|---|---|
| 1 | Site login | The '**Login**' method is called with 'username' (email is used as standard), 'password', 'ipAddress' and 'concurrencyCookie' parameters. | Credentials are authenticated and a 'UserToken' is included in the response where this succeeds.<br><br>This 'UserToken' is used in further interaction calls with the AD API. |

The 'concurrencyCookie' parameter must be a unique key value (e.g. a GUID) that is stored persistently as a cookie on the device where the login is being processed. The cookies should be set as follows:

- Name: 'ADConCtrlCookie'
- Value: [GUID value]
    - Note this GUID should be unique per device, not unique per username.
- Expiry: +1 year.

Full details of the request and response parameters for the Login method can be found in the AD API document.

**Reminder:** It is strongly recommended the 'PartyId' value returned by the 'AutoLogin' method is recorded for the user in your website's database if this has not previously been recorded. This is the user's unique identifier within Advance and can be used to directly link a website user record with a user stored within AD.

#### 6.2.2.3 Code sample to perform standard Login

Please find below a C# code sample written with help of AD Helper class.

```
NameValueCollection loginParameters = new NameValueCollection();
loginParameters.Add("username", userName);
loginParameters.Add("password", userPassword);
loginParameters.Add("ipAddress", ipAddress);
loginParameters.Add("concurrencyCookie", concurrencyCookie);

ADResult loginResult = InvokeADApi(ADResponseFormat.XML, loginUrl, ADMethod.POST, loginParameters);

if (loginResult.ResultCode == ADResultCode.OK)
{
        ADResultType resultType = (ADResultType)loginResult.Data.Items.FirstOrDefault(item =>
       item.Name == "LoginDetails");
       ADResultProperty userToken = (ADResultProperty)resultType.Items.FirstOrDefault(item =>
       item.Name == "UserToken");
       ADResultProperty partyID = (ADResultProperty)resultType.Items.FirstOrDefault(item => item.Name
       == "PartyID");

}

else

{

        ADHelper.LogErrorMessagesFromADResult(loginResult, string.Format("Error performing Login AD
        API call for user name {0}", userName));
        throw new ADAuthenticationException(loginResult.Messages);

}
```

## 6.3 AutoLogin process

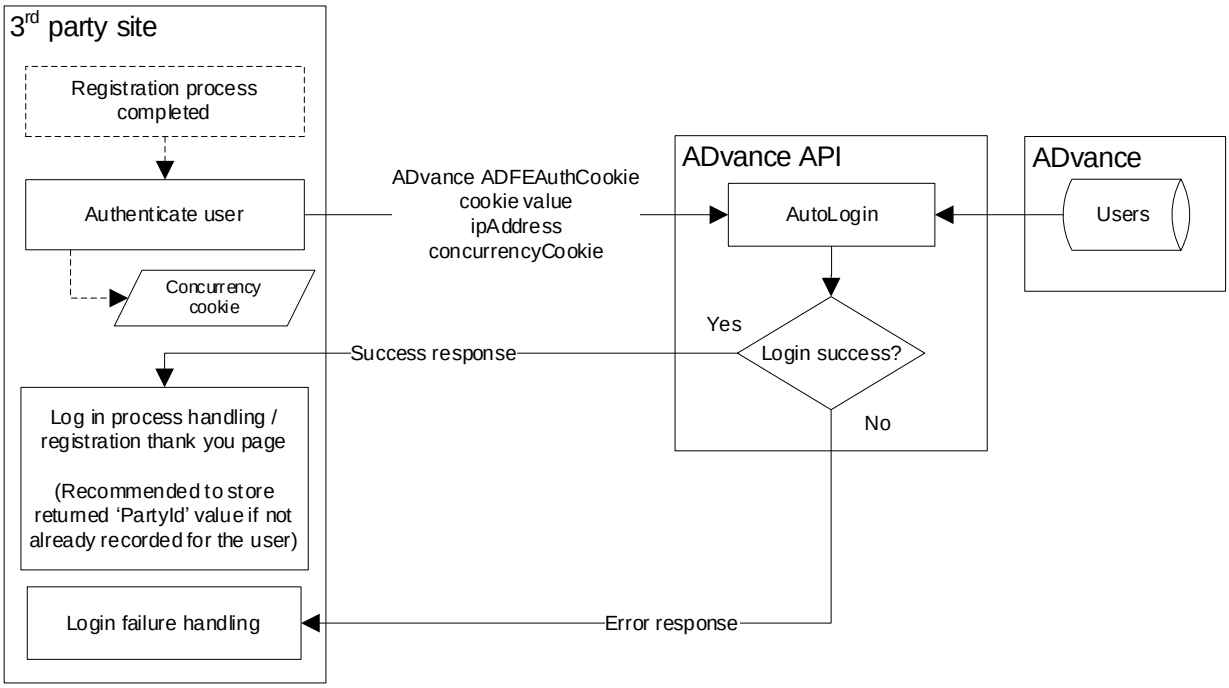
### 6.3.1 Introduction

This method is useful where the user is to be logged into the website following the completion of any of the following.

- AD registration process
- AD checkout process (which would have asked the user to register or sign in)
- The user has had to login whilst attempting to access an AD page, such as 'My Subscriptions' or the 'Edit user details' form
- Completion of the 'Redirect based Login process' (as per **Section 6.2.1**)

The AutoLogin method returns the user's 'UserToken' value which is required to complete further interactions with the AD API for the user.

### 6.3.2 Workflow diagram




Request and response values are as per the standard login method ('Login') detailed in **Section 6.2.2**.

Where the AutoLogin call is made following a registration which included a return URL parameter (retURL), the user will be returned to the URL after submitting their registration form. In this case any messaging required to be delivered to the user after registration would have to be included on the website page which the user is returned to.

**Reminder:** It is strongly recommended the 'PartyId' value returned by the 'AutoLogin' method is recorded for the user in your website's database if this has not previously been recorded. This is the user's unique identifier within Advance and can be used to directly link a website user record with a user stored within AD.

# 7 Email activation (optional)

Where enabled within AD, email activation is required following a user registration, or where an existing user modifies their email address.

The current activation status is returned in the response to AD's 'login' API methods (as detailed in **Section 6**):




It is the responsibility of the 3rd party frontend site to apply any alternative user journeys on the frontend site where a user is returned with an 'Activated' value of 'False'.

Activation is achieved by the user clicking a link within an AD generated activation email. The email can be sent to a user via the following AD API method:

- /User/SendRegistrationActivationEmail/1

The 'userToken' value returned by the AD login response must be included in the request to this method.

If registration occurs via an AD frontend registration form, the validation email will automatically be sent following the submission of the form. If AD frontend forms are not used, the above AD API method must be called by the 3rd party site following registration in order to generate the initial validation email to the user.

Changes applied to the email address of an existing AD party via AD frontend forms will also result in the validation email being automatically sent to the user. If AD party emails are modified directly via AD API interactions (i.e. not via AD forms), the above API method must be called in order for the validation email to be sent to the user.

# 8 Enabling single-sign-on with AD pages after a successful website login

This is to ensure that the user doesn't have to sign in twice when going from the website to AD pages. For example, user comes to your website and signs in (which is done by calling AD API). Now if they click on the link to the AD 'Edit user details' form, they should not be asked to sign in again. That said, some clients prefer their users to login again for security reasons especially before starting the checkout process.

The best to implement single-sign-on is by setting the SSO cookie in one of the following ways.

- SSO cookie set via a redirect to the AD cookie creation page
- SSO cookie set via JavaScript initiated redirect to the AD cookie page
- SSO cookie set on first access to an AD account page

### 8.1 SSO cookie set via a redirect to the AD cookie creation page

There is a secure option which requires you to go to AD after login and return, i.e. **https://(AD frontendURL)/adfetokenauth.ashx?token={usertoken}&returnUrl={returnurl}**

This will create an AD cookie in the AD domain, meaning that once the user goes to an AD page they will be already logged in into the AD domain pages.

### 8.2 SSO cookie set via JavaScript initiated redirect to the AD cookie page

It is also possible to call the AD cookie page page through AJAX. This method involves returning the following JavaScript to the page to force a page redirect (assuming the user has JavaScript enabled):

```
<script type='text/javascript'>
parent.location.href='https://(ADfrontendURL)/adfetokenauth.ashx?token={usertoken}&returnUrl={returnurl}'</script>
```

### 8.3 SSO cookie set on first access to an AD page

Similar to Option 1, this methods utilises a link to the AD cookie creation page but with the return URL parameter being set to the target AD page which the user is attempting to access. This enables the SSO cookie to be created at the time where the specific AD page needs to be accessed.

An example link to AD's edit user details page is as follows:

**https://(ADfrontendURL)/adfetokenauth.ashx?token={usertoken}&returnUrl=https%3A%2F%(ADfrontendURL)%2Fedituserdetails**

The return URL parameter ensures that after the redirect the user is correctly put back into the Edit user details form seamlessly signed in.

# 9 Requesting user details

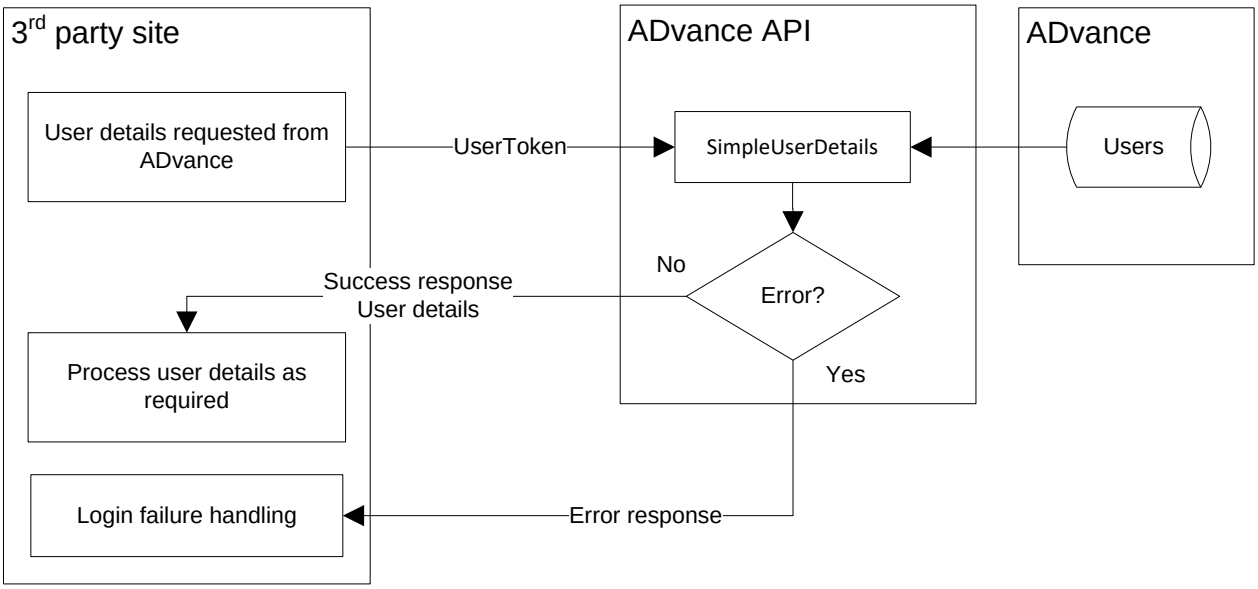
## 9.1 Introduction

An API method is available for the request of user details from AD. This may be useful to fetch user data following the completion of the registration process or another process flow where data needs to be retrieved.

**Abacus recommend retrieving user details at <u>every</u> sign in where these are stored locally on your website. This ensures that any changes to the user in AD are reflected on your website, for e.g. changes to name.**

The methods to request user data is reliant on a 'UserToken' being available to submit to the AD API. This is retrieved via either of the login methods detailed in **Section 6**.

## 9.2 Workflow diagram




## 9.3 SimpleUserDetails API

The AD API method is:

**https://{API-HOST}/User/SimpleUserDetails/1**

The 'UserToken' value is the only required field for this call, although other parameters are supported. Please refer to the AD API documentation for details of additional parameters as well as request and response formats.

## 9.4 Code sample to request user details

Please find below a C# code sample written with help of AD Helper class.

```
string userDetailsUrl = String.Concat(Convert.ToString(ConfigurationHelper.GetConfigSetting("AD",
"APIRoot")), "User/SimpleUserDetails/1");

NameValueCollection userDetailsParameters = new NameValueCollection();
userDetailsParameters.Add("UserToken", userToken.Value);
userDetailsParameters.Add("demographicsList",
Convert.ToString(ConfigurationHelper.GetConfigSetting("AD", "UserDetailsDemographicsList")));

ADResult userDetailsResult = InvokeADApi(ADResponseFormat.XML, userDetailsUrl, ADMethod.POST,
userDetailsParameters);

if (userDetailsResult.ResultCode == ADResultCode.OK)
{
        //ensure that a record exists in sub table
        Hashtable userRelatedData = ADHelper.GetUserRelatedDataFromAPIResponse(userDetailsResult);
        String email = userRelatedData["Email"];

}
else
{
        ADHelper.LogErrorMessagesFromADResult(autoLoginResult, string.Format("Error performing
        Autologin AD API call for user PartyId {0}", partyId));
        ADHelper.InvokeLogoutHandler();
}
```

# 10 Edit user details

## 10.1 Introduction

You should add a hyperlink to the AD 'Edit user details' form from your website. This link should only be accessible to logged in users.

## 10.2 Seamless access to AD 'Edit user details' page

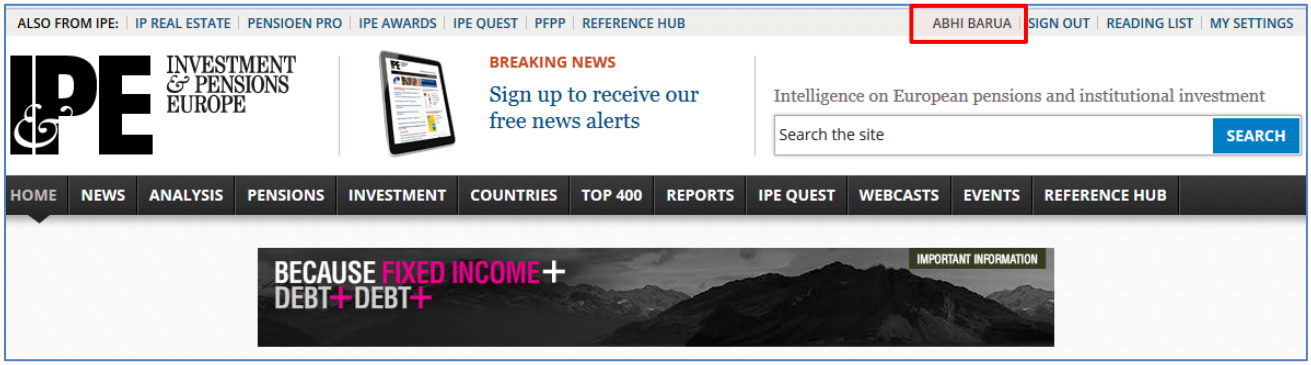
**Section 8** of this document details the methods which can be utilised to allow seamless access to AD pages you're your website without requiring the user to sign in to AD separately.

If you want users to sign in again, these methods can simply be omitted from the integration with a standard link to the AD 'Edit user details' page.

## 10.3 Updating local website database

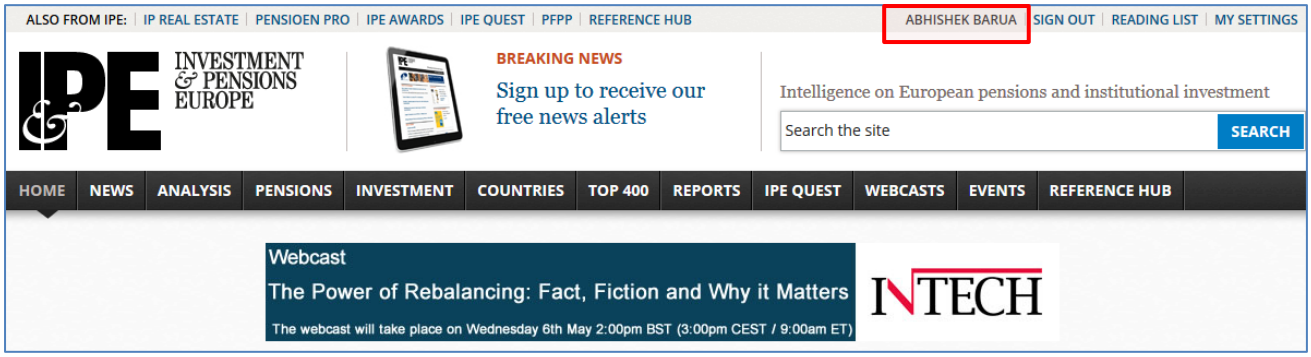
You may wish to execute a logout / login action in the background when the user comes back from the AD edit form to your website. This will ensure that any changes they made are reflected on your website.

For example, let's assume your website displays the name of the logged in visitor (i.e. Abhi Barua) in the website masthead, or against any comments they may have made on articles, and this is driven by the information you hold in your local website database.




The visitor then changes their name (i.e. Abhishek Barua) via the AD 'Edit user details' form.

Assuming that you have coded the sign in action to refresh your local database, you could execute a silent logout and login behind the scenes to ensure that the name in the masthead is displayed correctly.




# 11 Password recovery

## 11.1 Introduction

The password recovery mechanism differs depending on the configuration of your AD implementation.

| Approach | Comments | AD API to invoke |
|---|---|---|
| Send an email to the AD user with the human-readable/plain-text password | Non-encrypted passwords are held for the brand in AD | ForgottenPassword |
| Sending an email to the AD user with a password reset link | Encrypted passwords are held for the brand in AD | ResetPassword |

Only one approach can be enabled at a time for a particular brand and the preferred method must be specified at the start of the project.

### 11.1.1 Testing password recovery API integration

On AD TEST, we do not send out forgotten or reset password emails to the actual email address. Instead they are sent out to an email address that Abacus manage to avoid accidental spamming. If you are using the forgotten password or password reset APIs and want to test it, please inform us and we can arrange for the emails to be forwarded on to you.

## 11.2 ForgottenPassword API

### 11.2.1 Calling the API

The 'username' value is submitted to the API as a POST parameter. The API method is located at:

**https://{API-HOST}/User/ForgottenPassword/1**

Full details of the request and response formats for this API can be found in the AD API document.

### 11.2.2 Code sample to invoke the ForgottenPassword API

Please find below a C# code sample written with help of AD Helper class.

```
string accessHash = Convert.ToString(ConfigurationHelper.GetConfigSetting("AD", "APIAccessHash"));
string accessToken = Convert.ToString(ConfigurationHelper.GetConfigSetting("AD", "APIAccessToken"));
string forgottenPasswordUrl = String.Concat(Convert.ToString(ConfigurationHelper.GetConfigSetting("AD",
"APIRoot")), "User/ForgottenPassword/1");

NameValueCollection parameters = new NameValueCollection();
parameters.Add("UserName", Email);

ADResult forgottenPasswordResult = ADAPIInvoker.Invoke(accessHash, accessToken, ADResponseFormat.XML,
forgottenPasswordUrl, ADMethod.POST, parameters);

if (forgottenPasswordResult.ResultCode == ADResultCode.OK)
{
        ADResultType resultType = ((ADResultType)forgottenPasswordResult.Data.Items.FirstOrDefault(item =>
        item.Name == "ForgottenPasswordModel"));
        ADResultProperty partyID = ((ADResultProperty)resultType.Items.FirstOrDefault(item => item.Name ==
        "PartyID"));
        ADResultProperty newPassword = ((ADResultProperty)resultType.Items.FirstOrDefault(item =>
        item.Name == "Password"));

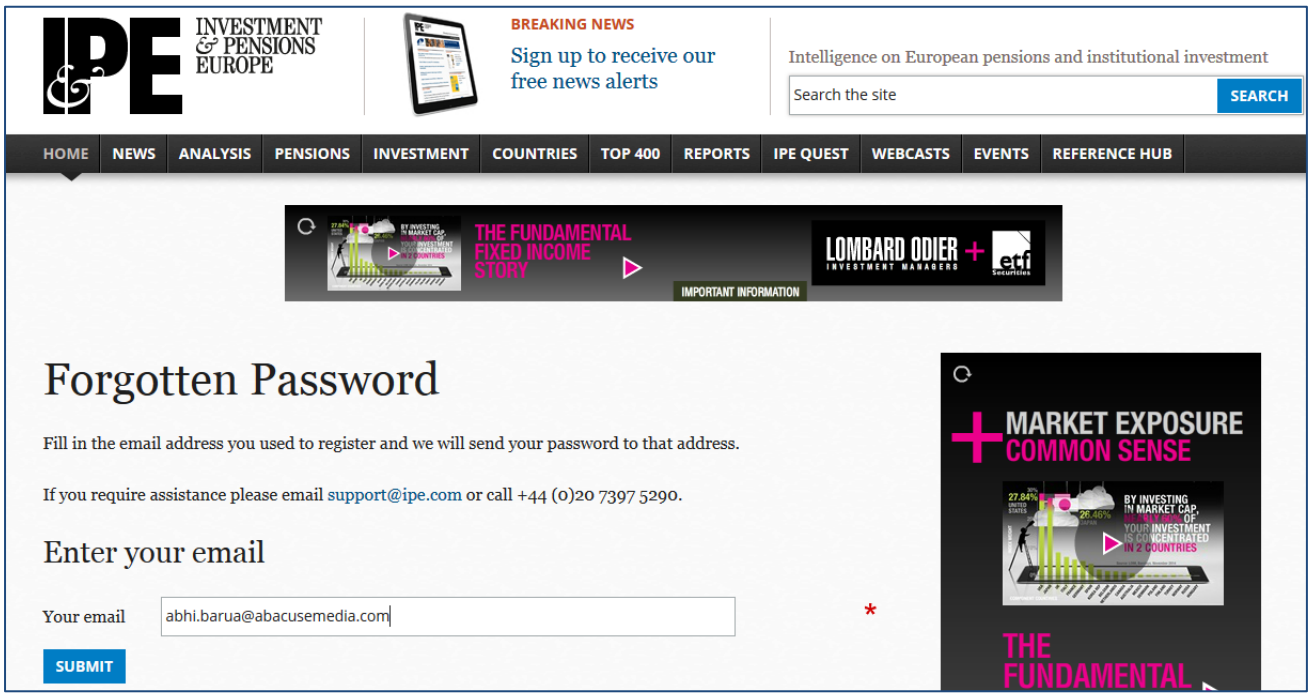
}
else
{

       //Email does not exist in our system or forgotten password email was not sent for
       other reasons (check results code)

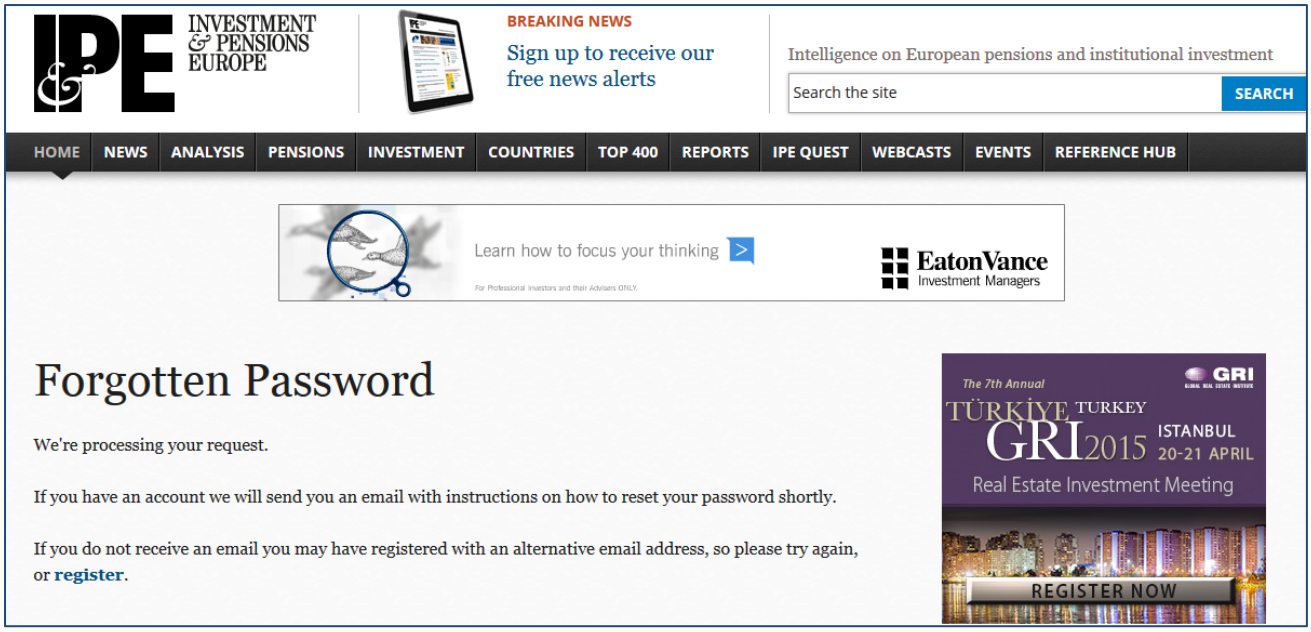
}
```

### 11.2.3 Frontend user experience

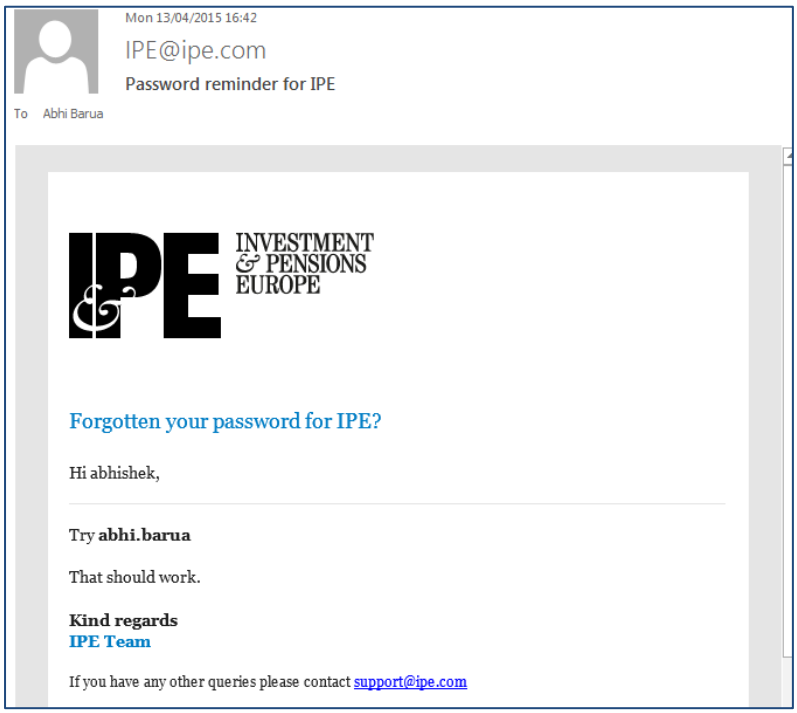
User requests forgotten password via a page on your website.




Confirmation message shown on your website.



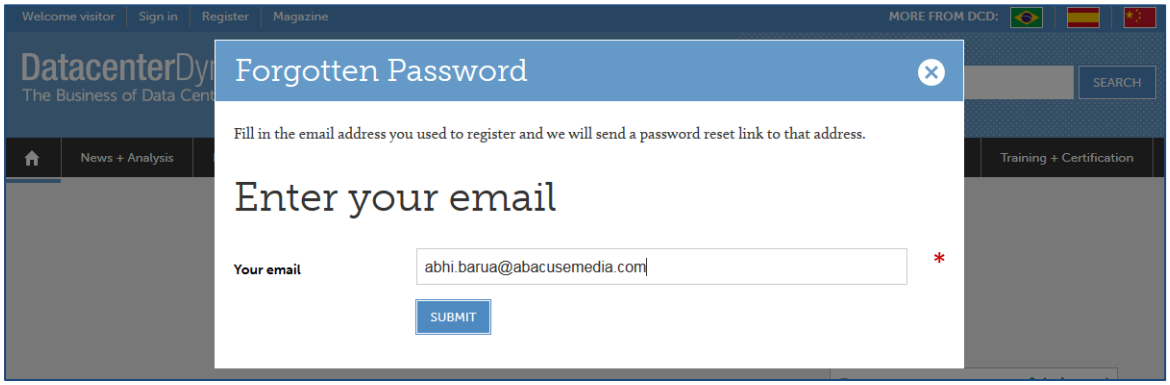

User receives an email with the password in it – this is generated by AD. Contents of the email can be customised by editing the relevant AD text asset – please ask AD Services for help.



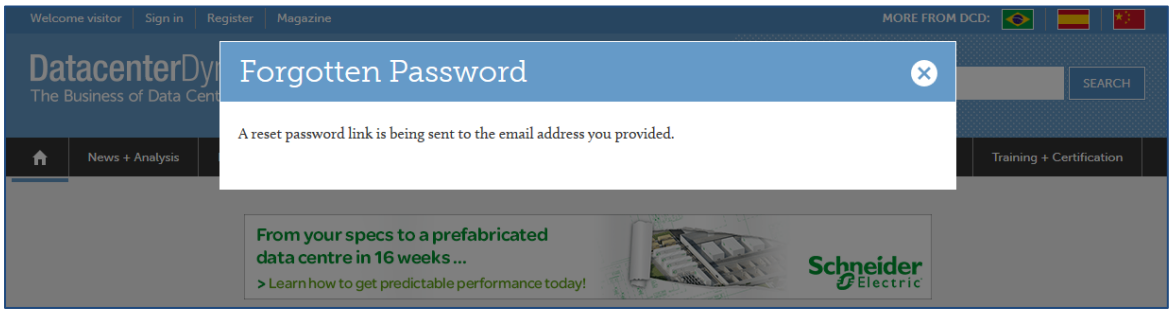

## 11.3 PasswordReset API

### 11.3.1 Frontend user experience

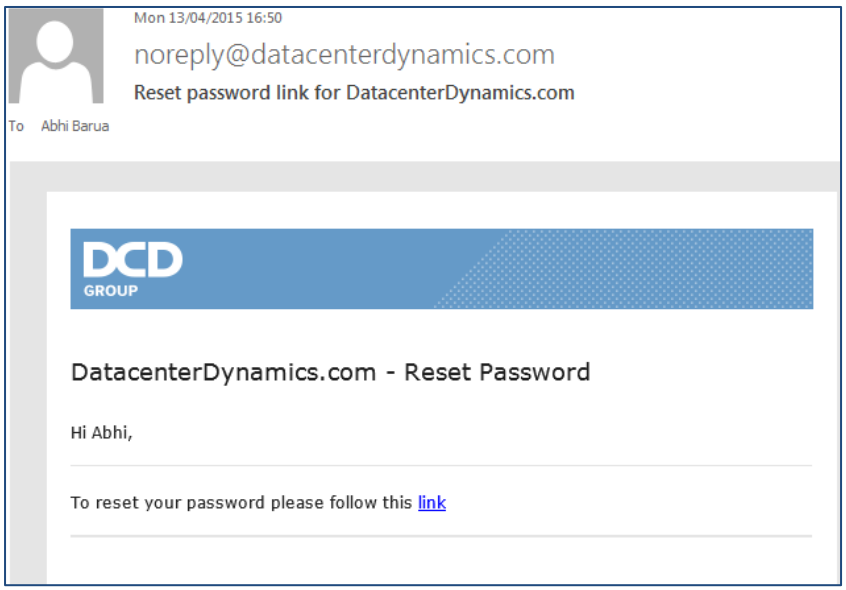
User wants to reset password via a page on your website.



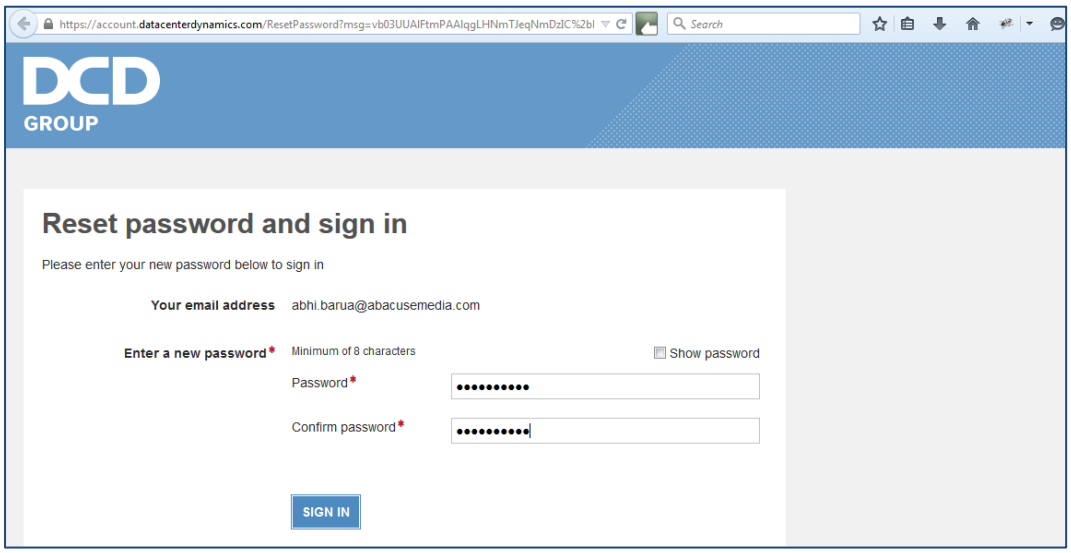

Confirmation message shown on your website.



User receives a reset password email – this is generated by AD. Contents of the email can be customised by editing the relevant AD text asset – please ask AD Services for help.




User clicks on the reset link in the email and is taken to an AD page. Contents of the page like heading and commentary can be customised by editing the relevant AD text asset – please ask AD Services for help.




User enters new password and is taken to the return URL specified. In most cases, this will be your website URL. Please ensure your website detects the 'ADFEAuthCookie' and logs the user into the website.

# 12 Implementing gatekeeping on premium online content

## 12.1 Introduction

Access control on articles in your website is best implemented by making gatekeeping calls to AD. This means that each time the user lands on a piece of gated content should make a gatekeeping call to AD. AD will check all access rules set up and respond with a “Allow access” or “Deny access” message.

Abacus do not recommend keeping a local copy of user entitlements and subscription details. AD allows the creation of highly customisable products and product offers. Attempting to copy this over to the local database to grant or deny access to frontend users is likely to be complicated, cumbersome and error-prone. Over time as new products and product offers are added or existing ones modified, the complexity will increase and there will have to be some form of hard-coding done to map product entitlements in AD to the equivalent entitlements in your system.

Therefore, our recommended approach is that you should make gatekeeping calls to AD for access control.

## 12.2 Requesting access for individual content items based on Metadata/category rules

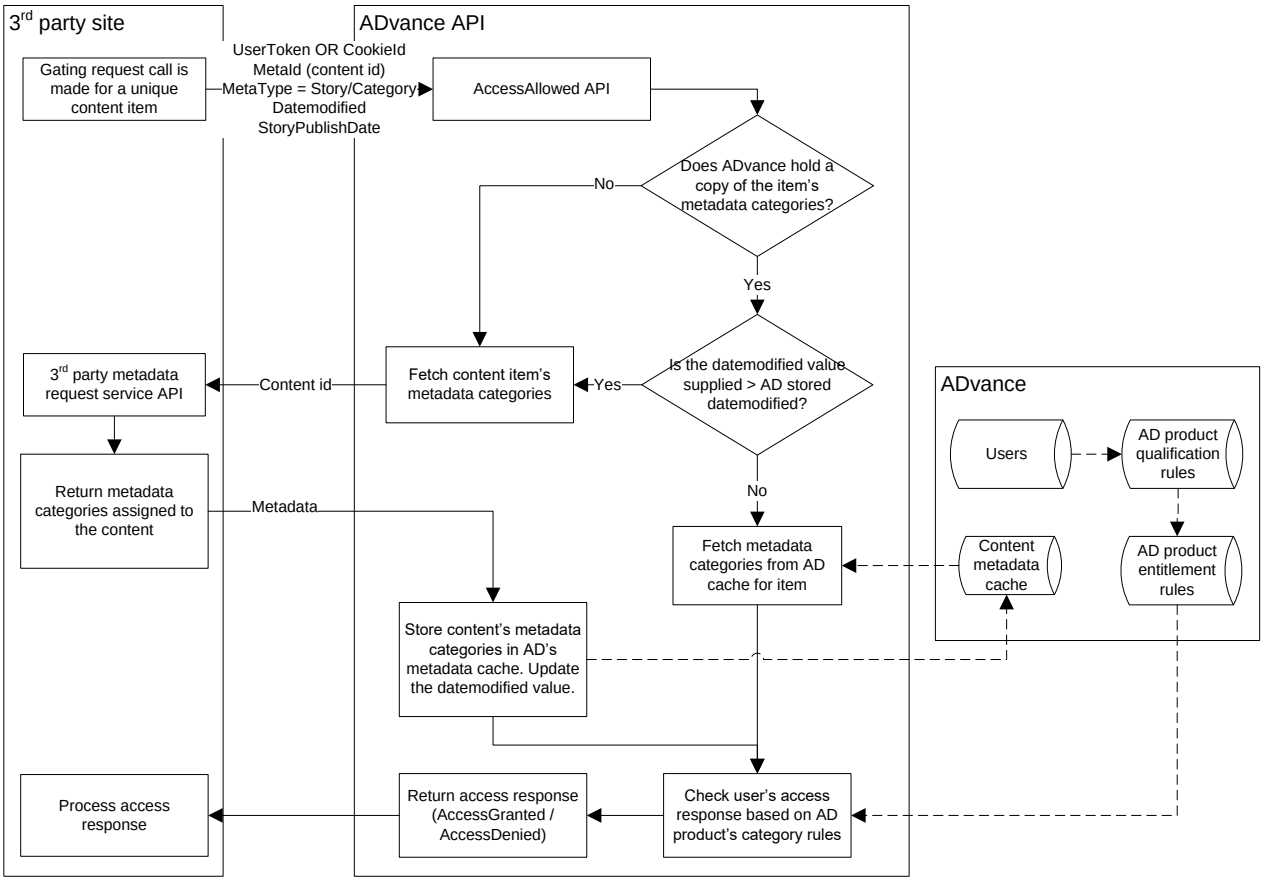
### 12.2.1 Introduction

This method allows for individual content items on your website to be queried against AD product rules to determine whether a user should be granted access to that content. These AD product rules are based on metadata (category) entitlements which are compared against the metadata associated with the content item on your website to decide on whether access should be given or not.

To support this implementation a Metadata request service must be available on your website so that Metadata (categories) can be requested by AD for the individual content items. The requirements for this interface must be discussed with Abacus prior to the implementation. A summary of the endpoints to be supplied to Abacus is provided in **Section 12.2.4**.

### 12.2.2 Workflow diagram

The workflow steps are as follows.




### 12.2.3 Calling the API

The gate keeping method is located at:

**https://{API-HOST}/GateKeeper/AccessAllowed/1**

Full details of this API method can be found in the AD API document: LINK

### 12.2.4 Supporting endpoints required to be provided by the non-Abacus content management platform

Two endpoints must be provided by the non-Abacus content management platform to support this method of access control:

#### 12.2.4.1 Category/taxonomy schema endpoint

This endpoint must provide the category/taxonomy schema to which article/story content will be associated with\in the non-Abacus content management platform. It will be used to present category/taxonomy items when configuring access control rules in the ADvance admin UI.

**Example endpoint URL:** https://**your.frontend.domain**/categorylist.aspx

**Example request and response:**

Request example (in this case provided by Abacus' Webvision Cloud CMS):
https://demo.webvisioncloud.com/ad/categorylist.aspx

**Response detail:**

The response should include all categories to be utilised for access control rules in ADvance. The items should represent any hierarchy present in the taxonomy structure.

| Item | Description |
|---|---|
| category[id] | A unique id for the taxonomy/category item. |
| portalcode | A unique id for the frontend site to which the taxonomy schema relates. If this is for a single frontend site, the value will be common for all items included. |
| category | The label of the taxonomy/category item. This will be displayed when configuring access control rules in the ADvance admin UI. |
| priority | This is not used for external schemas. Use value 0 for all items. |
| parent | The direct parent of the taxonomy/category item in the schema (which should also be present within the response). |
| parents | An ordered list of all ancestor items for the taxonomy/category item. Values are pipe (\|) delimited, with a pipe also included at the start and end of the string value. Items must be ordered 'top ancestor' to 'direct parent'. |
| datecreated | The datetime the category/taxonomy item was created in the schema. Values should be formatted as per the following example: `2015-03-23T20:38:16.663+00:00` |

#### 12.2.4.2 Article/story category associations

This endpoint must provide the category/taxonomy mapping for article/story items within the non-Abacus content management platform. It is used to allow the ADvance access control API ('/AccessAllowed') to determine the associated categories when an entitlement check is made using an article/story id value. The resulting category items are then evaluated against ADvance held access rules to determine the user's entitlements (granted or denied).

**Example endpoint URL:**
https://your.frontend.domain/ad/storycategorymapping.aspx?storycode=1003947&datefrom=20100501

**Example request and response:**

Request example (in this case provided by Abacus' Webvision Cloud CMS):
https://demo.webvisioncloud.com/ad/storycategorymapping.aspx?storycode=1003947&datefrom=20100501

**Request detail:**

| Item | Mandatory | Description |
|---|---|---|

| | | |
|---|---|---|
| storycode | No | A unique id for the article/story item for which the endpoint should return the related taxonomy/category data. The 'value must match that which is to be used in the ADvance API access entitlement check ('/AccessAllowed').<br><br>Where omitted all article/story items should be returned in the response. |
| datefrom | No | The value must filter the returned article/story items to those which have been created/updated on or after the specified date. |

**Response detail:**

The response should include all categories to be utilised for access control rules in ADvance. The items should represent any hierarchy present in the taxonomy structure.

| Item | Description |
|---|---|
| storycode[id] | A unique id for the article/story item being returned. |
| storycode[headline] | The title of the article/story item. |
| catcode | The unique id values of the taxonomy/category items related to the story. These should match those returned by the /categorylist.aspx endpoint. |

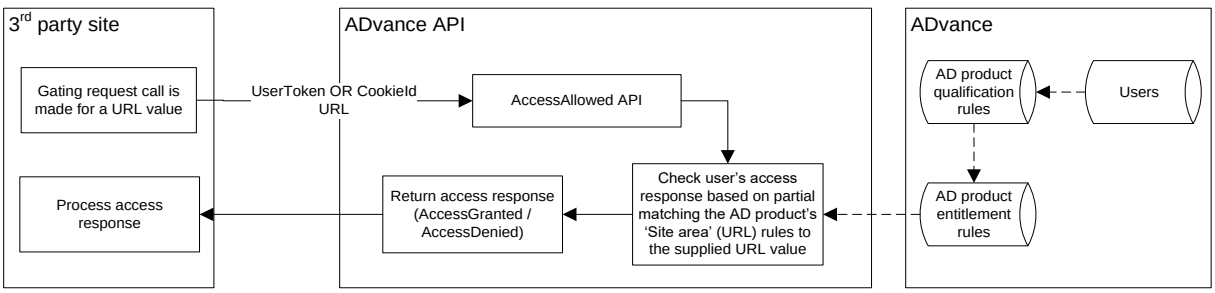
## 12.3 Requesting access based on URL of the item to be gated

### 12.3.1 Introduction

This method allows for URL based access control within the 3rd party site. The URL supplied for the API call is compared against the 'site area' rule values present for the user's products, returning an access granted response where any of these are a partial match.

### 12.3.2 Workflow diagram

The process steps are as follows:



### 12.3.3 Calling the API

The gate keeping method is located at:

**https://{API-HOST}/GateKeeper/AccessAllowed/1**

Full details of this API method can be found in the AD API document.

## 12.4 Code sample for making a gatekeeping call

Please find below a C# code sample written with help of AD Helper class.

```
NameValueCollection parameters = new NameValueCollection();
parameters.Add("UserToken", userToken);
parameters.Add("cookieId", cookieID);
parameters.Add("Url", Convert.ToString(HttpContext.Current.Items["UrlRewriter.NET.RawUrl"]));
parameters.Add("MetaId", Convert.ToString(aStory.ContentCode));
parameters.Add("MetaName", Convert.ToString(ADMetaType.Story));
parameters.Add("MetaType", Convert.ToString(ADMetaType.Story));
parameters.Add("ModifiedDate", DateTime.ParseExact(aStory.Status.SystemModified, new string[] {
"dd/MM/yyyy HH:mm:ss", "yyyy-MM-dd HH:mm:ss", "yyyy/MM/dd HH:mm:ss" }, CultureInfo.InvariantCulture,
DateTimeStyles.None).ToString("yyyy-MM-dd HH:mm:ss"));
parameters.Add("StoryPublishDate", aStory.Status.PublishedDateAsDate);

ADResult gateKeepingResult = ADAPIInvoker.Invoke(accessHash, accessToken, ADResponseFormat.XML,
gateKeepingUrl, ADMethod.POST, parameters);


if (gateKeepingResult.ResultCode == ADResultCode.OK)
{

ADResultType resultType = ((ADResultType)gateKeepingResult.Data.Items.FirstOrDefault(item => item.Name ==
"GKResult"));
ADResultProperty accessStatus = ((ADResultProperty)resultType.Items.FirstOrDefault(item => item.Name ==
"Status"));


if (accessStatus.Value == Convert.ToString(ADGateKeepingStatus.AccessDenied))

        //Access to resource denied

        else if (accessStatus.Value == Convert.ToString(ADGateKeepingStatus.AccessGranted))
                //Access to resource granted

        }
        else if (gateKeepingResult.ResultCode == ADResultCode.Error)
        {
                //An error occurred during the call

        }

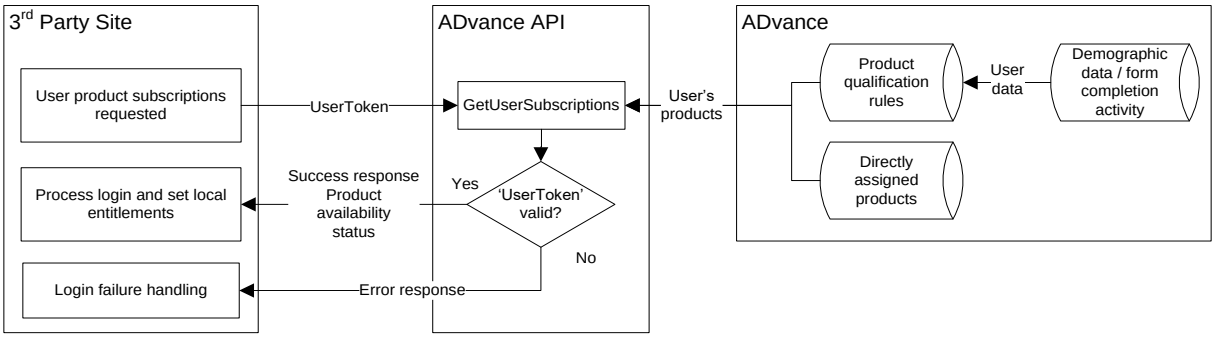
}
```

## 12.5 Retrieving details about a user's subscription products

### 12.5.1 Introduction

Although it is not recommended that user entitlements and gatekeeping are managed outside of AD's access control mechanism, it is possible to request a user's subscription products via the AD API.

### 12.5.2 Workflow diagram




### 12.5.3 Business rules

Please talk to Abacus's AD Services team for any specific queries.

- All live subscriptions are returned including complimentary and controlled circulation
- Expired subscriptions that are within renewal range are returned too (using lapsed period setting that is being set via AD Backend, under Configuration/Subscriber settings)
- The response includes both OrderStatus and SubscriptionStatus (as order may be live, but subscription may be expired)

### 12.5.4 Calling the API

The method is located at:

**https://{API-HOST}/User/GetUserSubscriptions/1**

Full details of this API method can be found in the AD API document.

# 13 Access denied messaging

## 13.1 Introduction

An access denied page (also called a 'barrier' page) is displayed to users trying to access content to which they don't have access.

AD does not have a messaging engine at present. Therefore, the contents and presentation of access denied message must be managed by your website.

## 13.2 Gatekeeping levels

The following levels are supported in Advance.

| Level | Comments |
|---|---|
| Free | User can view gated content anonymously |

| | |
|---|---|
| Registered | User must register to view gated content |
| Data gathering | User must fill in an Advance capture form to view gated content |
| Data filter | User must meet specific criteria as stated in a BI filter within Advance, for e.g. based on their demographics or subscription status, to view gated content |
| Subscription | User must hold a subscription to this product to view gated content |

One or more levels can be applied at the same time to achieve a tiered/metered access model. Please speak to us if this is a requirement and we can help set up some sample products for you to test this.

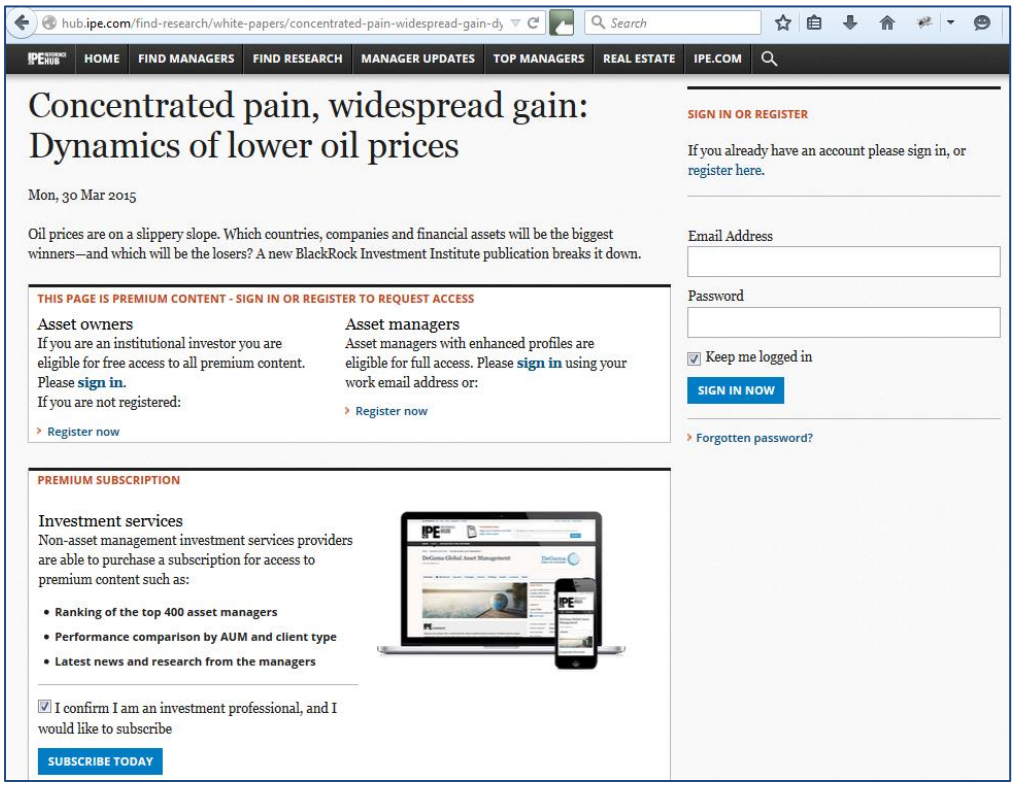
Advance gatekeeping API will simply return a “User has access” / “User does not have access” response and it is up to the website to display an appropriate access denied message.

For data-gathered products, i.e. those that require the user to fill in a form before being allowed to access content, the ID of the form to be filled will also be returned by the gatekeeper API. You should insert a link to the form in the access denied message.

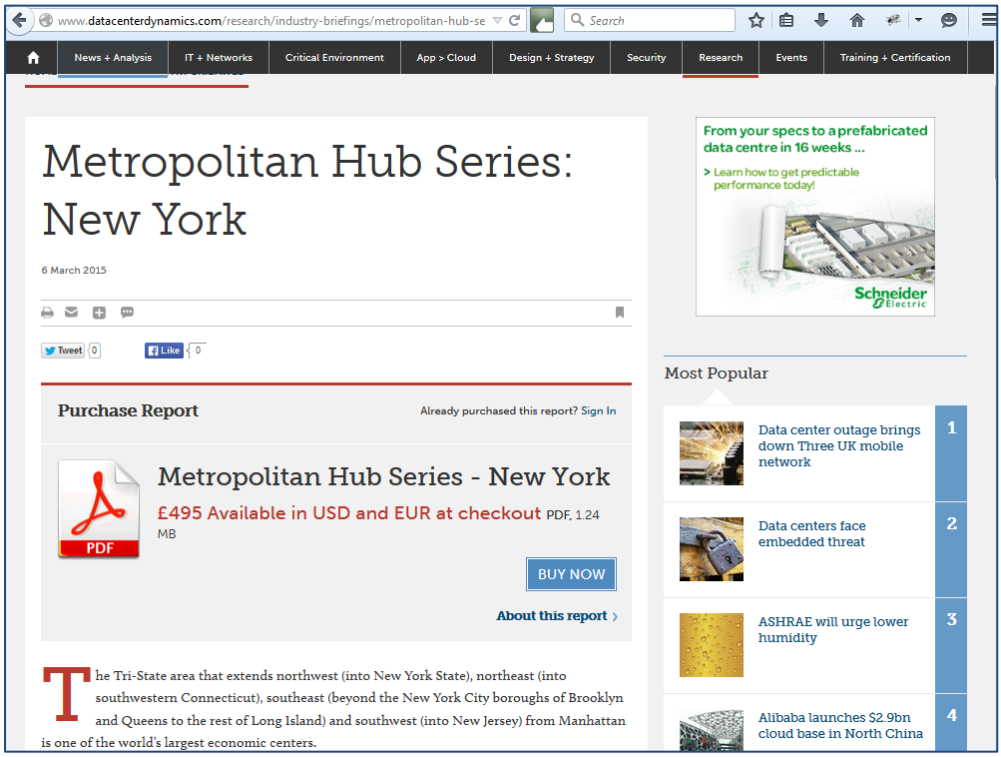
## 13.3 Frontend user experience on your website

Below are some examples of how some of our clients have done this – please talk to AD Services team to discuss any specific requirements you may have or if you need our advice on something.

### 13.3.1 Example 1



### 13.3.2 Example 2



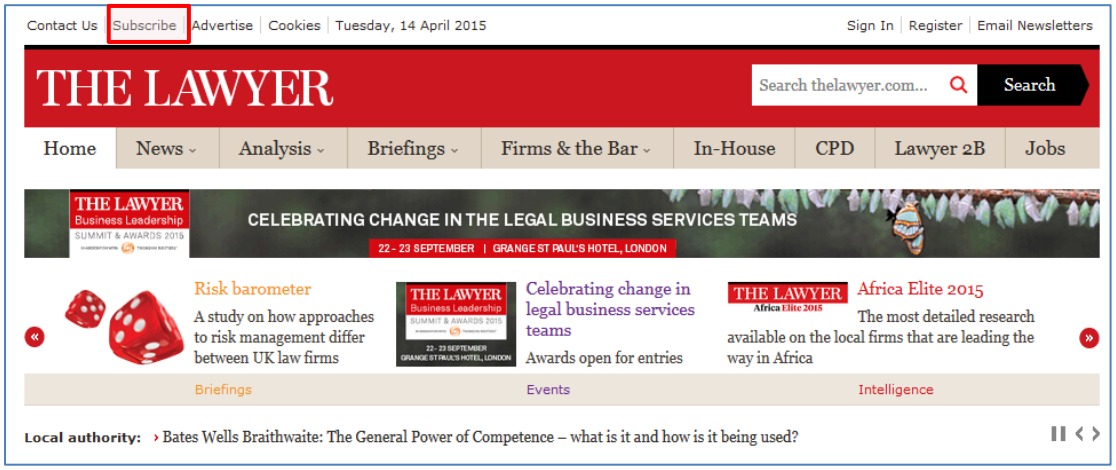

# 14 Selling subscriptions on your website using AD

## 14.1 Introduction

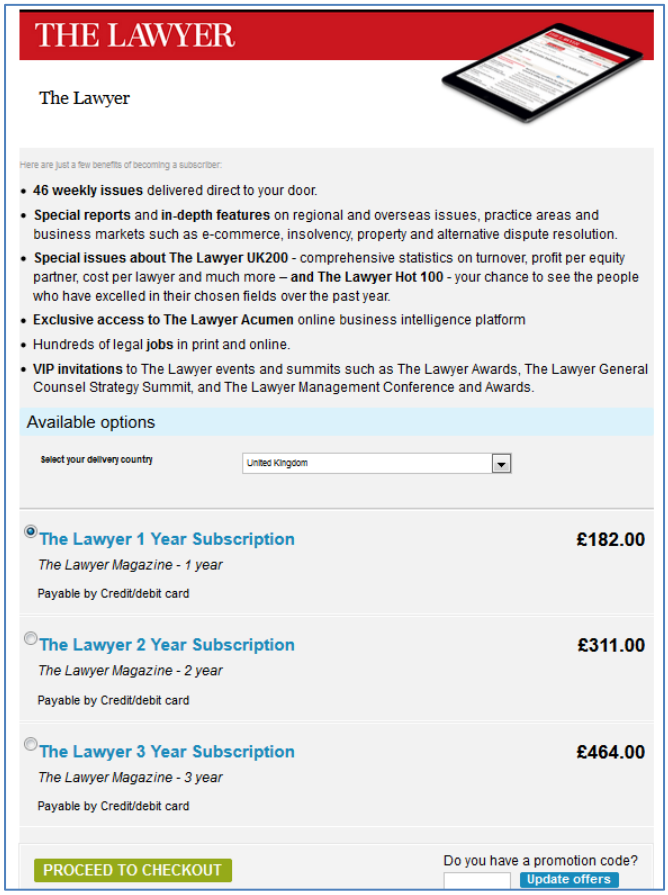
AD has a set of frontend checkout pages which are designed to allow you to sell subscriptions on your website. The end user can go through the checkout process without the need for you to manage this individually. Orders can also be placed through the AD backend.

## 14.2 Linking out to AD subscription pages

To direct users to our subscription pages, simply insert a link from our website to the "Product Offers" page – URL examples are in **Section 2.1**.



The “Product Offers” page, i.e. https://ad.thelawyer.com/subscribe, lists the subscriptions available in the chosen country (see the delivery country dropdown). You can create bundles involving different products like print, online, digital and events, and specify the duration of the subscription, pricing, tax etc. AD is a highly configurable system. Please talk to the AD services team to set up products, variants and offers.




## 14.3 Subscription / checkout steps

The checkout process comprises the following steps. You will be able to customise the messaging on these pages and on the confirmation email; please speak to AD services team for guidance and ensure that the text assets are correctly populated.

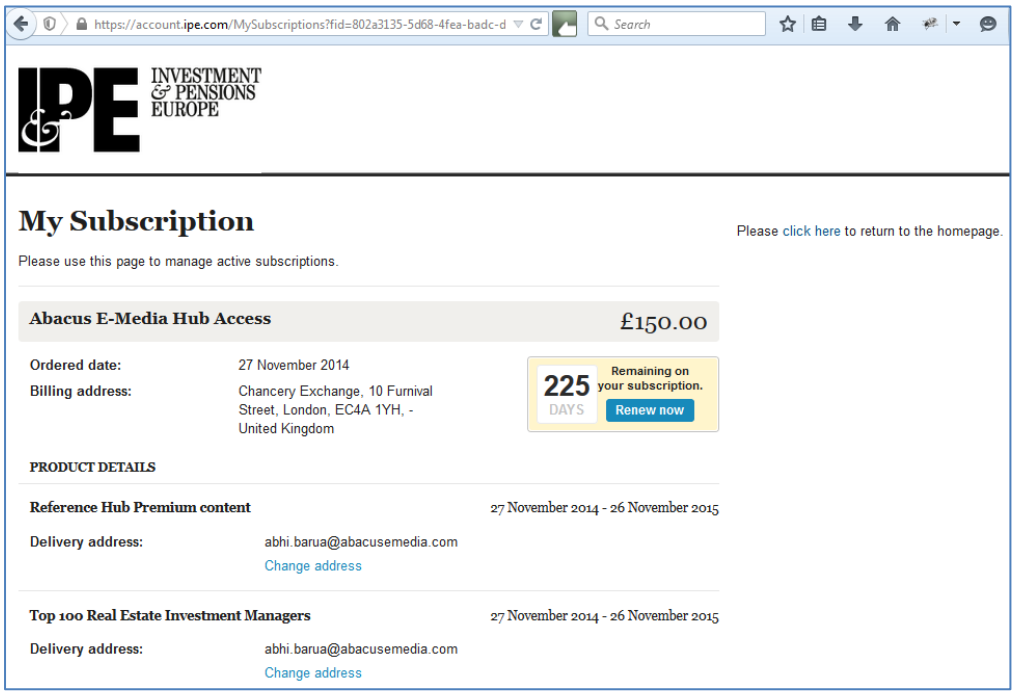
- Choose the subscription from the Product Offers page
- Login or register[1]
- Enter billing address
- Enter delivery address
- Enter payment details[2]
- Review order
- Order confirmation page[3] and confirmation email

[1] To ensure that new users about to make a purchase are not bombarded with too many questions, we recommend keeping the registration form light to collect only the bare minimum details like first name, last name, email and password. AD offers a special type of configurable light registration form designed especially for this purpose.

[2] Only one can be enabled at a time. Please contact Abacus for details of the supported payment providers.

[3] You can also display a data gathering form at the end of the checkout process, i.e. after the purchase has been made. Please speak to the AD services team if you wish to set this form up.

AD also offers a "My Subscriptions" where a user can see a list of the subscription they have. Like the other pages, you can simply link out to this page. Please ensure this link is only available to logged in users. For example, https://account.ipe.com/MySubscriptions




## 14.4 Website integration

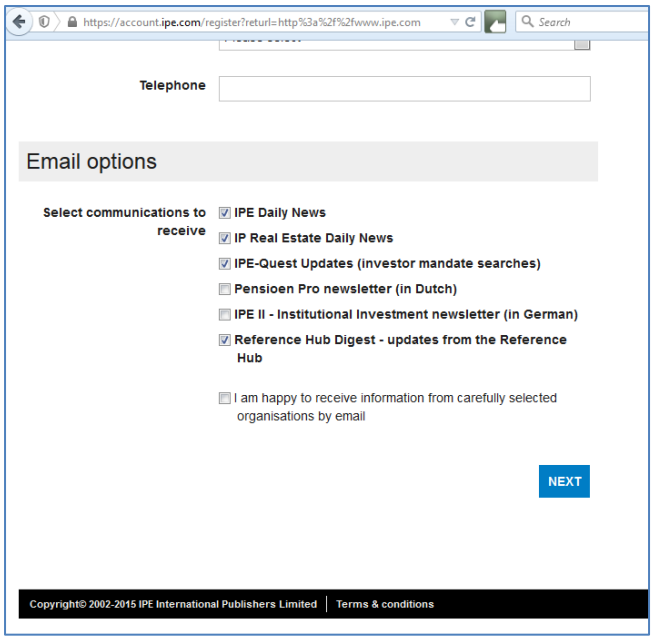
As far as website integration is concerned, you only need to direct users to the subscription page with a return URL specified. At the end of the purchase process we will send the user back to the return URL you specify.

You will need to have code to handle those users when they come to your website. At the end of the subscription process, the user will be in a logged in state on AD pages and may not be necessarily signed in on your website. This might be true if the user landed on the Product Offers page responding to an email marketing campaign that you may have sent. Therefore, you need to detect 'ADFEAuthCookie' cookie and carry out an auto-login action. As part of this you need to check if the user already exists on your local database. If he/she does, you should update their record with the latest information fetched from AD and if they don't you should create a local stub record saving the AD PartyId on your database.

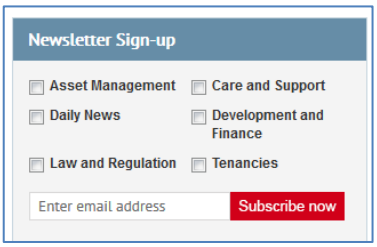
Another thing you may consider so as to enhance the user experience is to put the relevant return URL in. For example, if the user was viewing an article when being prompted to buy a subscription, you would ideally want the user to come back to the same article after completing the purchase. The code to construct the appropriate return URL falls under your remit.

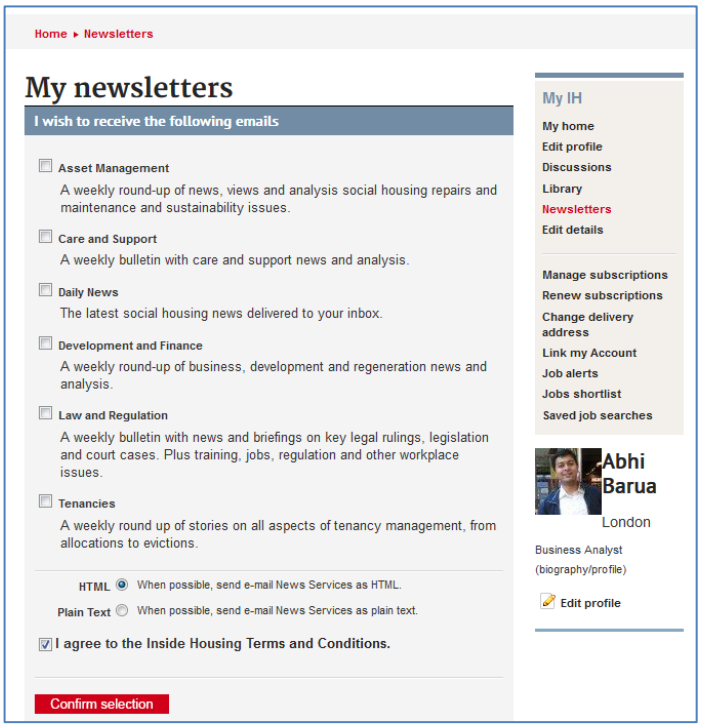
# 15 Managing newsletter sign-ups

Newsletter preferences can be managed via AD capture forms by adding the AD newsletter component to it. This can be done as part of new user registration and then via the Edit user details capture form.




Alternatively, you can also make a call to an AD API to fetch a list of available newsletters for that brand and display it using on your website. This approach allows you to configure the functional and display rules exactly how you would like it. For example, some of our clients display a newsletter widget that leads on to a dedicated newsletters page. The implementation is entirely up to you.




AD can also be integrated with 3rd party email service providers like Dotmailer, Adestra, CheetahMail etc. Additional costs will apply and these will vary based on the level of integration required, i.e. fully automated or a regular FTP-based dump. Please talk to your Account/Project Manager for more details.

Bounce-backs and campaign statistics (opens, click-throughs etc.) should be managed by your email provider.

# 16 Other user messaging options available to you

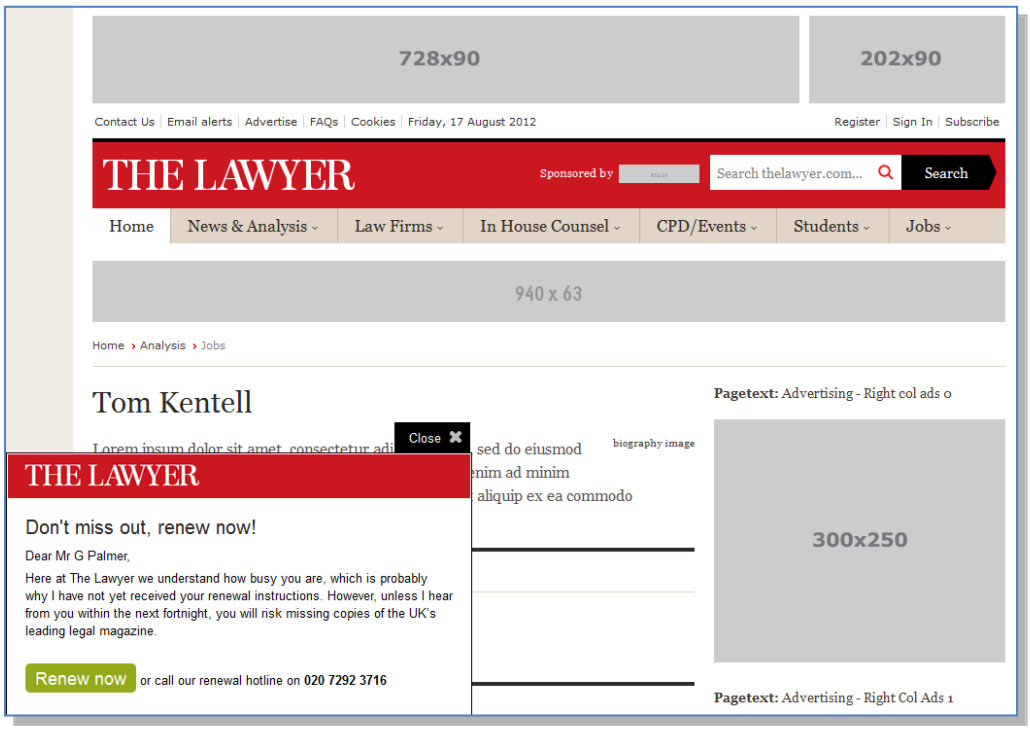
Please feel free to discuss with your Account/Project Manager any specific messaging needs you may have and we can advise if there is an out-of-the-box way to implement them.

## 16.1 Frontend online renewal messaging

You may want to display a message to users who are nearing the end of their subscription when they visit your website and provide them with incentives to renew. The user must be logged in for this to work.

Renewal messaging can be pushed out from the AD system to your website based on the renewal campaigns you have set up in AD. Please talk to AD Services team to know more about renewal campaigns.

Assuming the renewal campaigns are properly set up, the implementation of frontend renewal messaging is fairly straightforward. All you need to do is insert some JavaScript provided by Abacus into your website and we will do the rest.




### 16.2 Other ad-hoc messaging

There are ways in which you can display custom messages to logged in users, based on criteria like user demographics (custom attributes) or subscription statuses.

For example, if you want to display a promo message to all Lawyers based in London who receive the monthly print magazine so as to sell an online subscription, you should do the following.

- Set up a BI filter in AD with the required criteria
- Website should call the 'Does User Belong To BI Filter' AD API to determine if the logged in user meets the criteria
- If yes, website should be coded to display the required message in the preferred style

## 17 Appendix: AD call method helper classes

Below are classes that are required for the AD API method call to work.

### 17.1 ADResult

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Xml;

namespace CoreTemplates
{
    /// <summary>
    /// Represents result from AD API call
    /// </summary>
```

```
public class ADResult
{
    private List<ADResultMessage> _messages;

    /// <summary>
    /// Default constructor
    /// </summary>
    public ADResult()
    {
        this.ResultCode = ADResultCode.Error;
        _messages = new List<ADResultMessage>();
        Version = 1;
        Data = new ADResultType();
    }

    /// <summary>
    /// API Result version
    /// </summary>
    public int Version { get; set; }

    /// <summary>
    /// Result code
    /// </summary>
    public ADResultCode ResultCode { get; set; }

    /// <summary>
    /// List of result messages
    /// </summary>
    public List<ADResultMessage> Messages
    {
        get
        {
            return _messages;
        }
    }

    private ADResultNamedItem _data;

    /// <summary>
    /// Actual result object. Must be XML serialisable
    /// </summary>
    public ADResultType Data
    {
        get
        {
            return _data as ADResultType;
        }
        set
        {
            _data = value;
        }
    }

    public ADResultSequence Sequence
    {
        get
        {
            return _data as ADResultSequence;
        }
        set
```

```
            {
                _data = value;
            }
        }

        /// <summary>
        /// The first item in collection is the actual return model
        /// </summary>
        public ADResultType FirstData
        {
            get
            {
                return (ADResultType)(this.Data.Items.FirstOrDefault());
            }
        }
        /// <summary>
        /// Generates an object from its XML representation.
        /// </summary>
        /// <param name="reader">The <see cref="T:System.Xml.XmlReader"/> stream from which
the object is deserialized.</param>
        public void ReadXml(XmlReader reader)
        {
            reader.ReadStartElement("ad"); // goes to response

            if (reader.LocalName != "response")
                throw new Exception("Invalid ADResult XML");

            //reader.ReadStartElement("response");
            this.Version = Convert.ToInt32(reader.GetAttribute("version"));

            reader.ReadStartElement(); // status

            if (reader.LocalName != "status")
                throw new Exception("Invalid ADResult XML");

            reader.ReadStartElement(); //code

            if (reader.LocalName != "code")
                throw new Exception("Invalid ADResult XML");

            switch(reader.ReadString())
            {
                case "OK":
                    this.ResultCode = ADResultCode.OK;
                    break;
                case "WARNING":
                    this.ResultCode = ADResultCode.Warning;
                    break;
                case "ERROR":
                    this.ResultCode = ADResultCode.Error;
                    break;
                default:
                    throw new Exception("Cannot deserialize AD Result, unknown status");
            }

            reader.ReadEndElement(); // code

            // pointing at messages now
            if (!reader.IsEmptyElement)
            {
```

```
            reader.ReadStartElement(); // messages

            while (true)
            {
                if (reader.NodeType == XmlNodeType.Element)
                {
                    if (reader.LocalName != "msg")
                        throw new Exception("Invalid ADResult XML");

                    ADResultMessage msg = new ADResultMessage();

                    switch (reader.GetAttribute("type"))
                    {
                        case "info":
                            msg.MessageType = ADResultMessageType.Info;
                            break;
                        case "warn":
                            msg.MessageType = ADResultMessageType.Warning;
                            break;
                        case "error":
                            msg.MessageType = ADResultMessageType.Error;
                            break;
                    }

                    msg.Code = Int32.Parse(reader.GetAttribute("code"));
                    msg.Message = reader.ReadString();
                    this.Messages.Add(msg);
                    reader.ReadEndElement(); // this must be closing msg node
                }
                else if (reader.NodeType == XmlNodeType.EndElement)
                {
                    reader.ReadEndElement(); // this must be closing messages node
                    break;
                }
                else
                    throw new Exception("Unexpected XML structure");
            }
        }
        else
            reader.ReadStartElement(); // messages, empty node, skips immediately to next
one

        reader.ReadEndElement(); // status
        this.Data = ReadType(reader); // read 'data' element
    }

    private ADResultProperty ReadProperty(XmlReader reader)
    {
        if (reader.NodeType == XmlNodeType.Element && reader.LocalName == "prop")
        {
            string propName = reader.GetAttribute("name");
            string propValue = string.Empty;

            if (!reader.IsEmptyElement)
            {
                reader.Read();

                if (reader.NodeType == XmlNodeType.Text)
                    propValue = reader.ReadString();
```

```
                    // skip everything until the end of this item
                    while (reader.NodeType != XmlNodeType.EndElement)
                        reader.Read();

                    // read that end now and return
                    reader.ReadEndElement();
                }
                else
                    // this should read past empty element
                    reader.Read();

                ADResultProperty prop = new ADResultProperty(propName, propValue);

                return prop;
            }
            else
                return null;
        }

        private ADResultSequence ReadSequence(XmlReader reader)
        {
            if (reader.NodeType == XmlNodeType.Element && reader.LocalName == "seq")
            {
                ADResultSequence seq = new ADResultSequence(reader.GetAttribute("name"));
                // go to the next node which should be first type

                if (reader.Read())
                {
                    if (reader.NodeType == XmlNodeType.Element)
                    {
                        if (!reader.IsEmptyElement)
                        {
                            ADResultType type = ReadType(reader);

                            while (type != null)
                            {
                                seq.Add(type);
                                type = ReadType(reader);
                            }
                        }
                    }
                    else if (reader.NodeType == XmlNodeType.EndElement)
                        // the end of story
                        reader.ReadEndElement();
                }
                return seq;
            }
            else
                return null;
        }

        private ADResultType ReadType(XmlReader reader)
        {
            if (reader.NodeType == XmlNodeType.Element && (reader.LocalName == "type" ||
reader.LocalName == "data") )
            {
                ADResultType type = new ADResultType(reader.GetAttribute("name"));

                bool canRead = reader.Read();
```

```
                while (canRead)
                {
                    if (reader.NodeType == XmlNodeType.Element)
                    {
                        ADResultProperty prop = ReadProperty(reader);

                        if (prop != null)
                            type.AddProperty(prop);
                        else
                        {
                            ADResultSequence seq = ReadSequence(reader);

                            if (seq != null)
                                type.AddSequence(seq);
                            else
                            {
                                ADResultType t = ReadType(reader);

                                if (t != null)
                                    type.AddType(t);
                                else
                                    throw new Exception("Cannot recognise datatype in XML
deserialisation for API");
                            }
                        }
                    }
                    else if (reader.NodeType == XmlNodeType.EndElement)
                    {
                        // end of type
                        reader.ReadEndElement();
                        break;
                    }
                }
                return type;
            }
            else
                return null;
        }

        /// <summary>
        /// Converts an object into its XML representation.
        /// </summary>
        /// <param name="writer">The <see cref="T:System.Xml.XmlWriter"/> stream to which the
object is serialized.</param>
        public void WriteXml(XmlWriter writer)
        {
            throw new Exception("Serialisation not supported here");
        }
    }
}
```

## 17.2 ADResponseFormat

```
using System;

namespace CoreTemplates
{
    public enum ADResponseFormat
    {
```

```
        XML = 0,
        JSON = 1
    }
}
```

## 17.3 ADMethod

```
using System;

namespace CoreTemplates
{
    public enum ADMethod
    {
        GET = 0,
        POST = 1
    }
}
```



abacus

Abacus Software Ltd trading as Abacus No. 3387095
Registered office 21 Southampton Row, London WC1B 5HA

PCI DSS COMPLIANT · CYBER ESSENTIALS · bacs · Microsoft CERTIFIED · ppa · MemberWise · ISO 9001 and 27001 certified